

CLIO: Enabling automatic compilation of deep learning pipelines across IoT and Cloud

Jin Huang^{*}, Colin Samplawski^{*}, Deepak Ganesan^{*}, Benjamin Marlin^{*}, Heesung Kwon[†]

^{*}University of Massachusetts Amherst, [†]US Army Research Laboratory
{jinhuang, csamplawski, dganesan, marlin}@cs.umass.edu, heesung.kwon.civ@mail.mil

ABSTRACT

Recent years have seen dramatic advances in low-power neural accelerators that aim to bring deep learning analytics to IoT devices; simultaneously, there have been considerable advances in the design of low-power radios to enable efficient compute offload from IoT devices to the cloud. Neither is a panacea — deep learning models are often too large for low-power accelerators and bandwidth needs are often too high for low-power radios. While there has been considerable work on deep learning for smartphone-class devices, these methods do not work well for small battery-powered IoT devices that are considerably more resource-constrained.

In this paper, we bridge this gap by designing a continuously tunable method for leveraging both local and remote resources to optimize performance of a deep learning model. **Clío** presents a novel approach to split machine learning models between an IoT device and cloud in a *progressive* manner that adapts to wireless dynamics. We show that this method can be combined with model compression and adaptive model partitioning to create an integrated system for IoT-cloud partitioning. We implement **Clío** on the GAP8 low-power neural accelerator, provide an exhaustive characterization of the operating regimes where each method performs best and show that **Clío** can enable graceful performance degradation as resources diminish.

CCS CONCEPTS

• Computer systems organization → Cloud computing.

KEYWORDS

edge computing, cloud computing, computation off-loading, deep neural networks

ACM Reference Format:

Jin Huang^{*}, Colin Samplawski^{*}, Deepak Ganesan^{*}, Benjamin Marlin^{*}, Heesung Kwon[†]. 2020. CLIO: Enabling automatic compilation of deep learning pipelines across IoT and Cloud. In *The 26th Annual International Conference on Mobile Computing and Networking (MobiCom '20)*, September 21–25, 2020, London, United Kingdom. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3372224.3419215>

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from [permissions@acm.org](https://permissions.acm.org).

MobiCom '20, September 21–25, 2020, London, United Kingdom

© 2020 Association for Computing Machinery.

ACM ISBN 978-1-4503-7085-1/20/09...\$15.00

<https://doi.org/10.1145/3372224.3419215>

1 INTRODUCTION

While traditional IoT and sensor networks have focused on connecting simple sensors (e.g., thermostats, humidity sensors, etc), we are seeing a shift towards richer, always-on sensors (e.g. cameras, microphones, IMUs) which have become considerably more energy-efficient in recent years [9, 18, 35, 55]. In turn, these capabilities are driving more sophisticated applications of battery-powered sensors (e.g. home security, mobile health).

These trends have led to an emerging marketplace for associated compute elements that support continuous analytics at low power consumption. Several recently announced neural accelerator chipsets such as GreenWaves GAP8 [51], EtaCompute Tensai [50] and Syntiant NDP [52]) consume less than 100mW, thereby allowing us to envisage Mote-class devices equipped with neural accelerators.

Despite these developments, there remains a substantial gap between available resources on low-power accelerators and resources needed for efficient execution of state-of-art deep learning models. While model compression methods have been successful at squeezing large models into embedded devices like smartphones and Raspberry Pis, low-power accelerators represent a huge step up since they have two to three orders of magnitude less resources than these embedded processors (e.g. the GAP8 has a convolutional accelerator with 1 GFlops whereas Raspberry Pi has 24 GFlops and iPhone X has 350 GFlops). The successes of most model compression methods to date has been with relatively small compression ratios (e.g. less than 2× in compute and memory footprint without significant loss of accuracy [17]) but fitting even the smallest versions of state-of-art deep learning models like ResNet-18, VGG-16, and MobileNetV2 into an IoT device requires a 5–20× compression in compute and memory footprint, which is beyond the capability of existing methods. As a result, existing work on optimizing machine learning models for IoT has used simpler models and relied on hand-crafted optimizations rather state-of-art deep learning models and automated model compression techniques [26]. With either approach, the price is a substantial reduction in prediction accuracy due to the extreme levels of compression required to fit the model into available resources.

An alternative to model compression is to leverage wireless communication for transmitting data to the cloud for processing. Rather than compressing the model, we can instead compress the raw data using lossy compression techniques, transmit it to the cloud, and use cloud resources for prediction. This is particularly attractive since the efficiency of low-power radio front-ends and their duty-cycling efficiency has steadily improved over the last two decades (e.g. BLE, Zigbee, etc [1, 36, 49]). However, the challenge is that low-power radios have limited bandwidth and are also sensitive

CLIO: 实现跨物联网和云的深度学习管道的自动编译

Jin Huang^{*}、Colin Samplawski^{*}、Deepak Ganesan^{*}、Benjamin Marlin^{*}、Heesung Kwon[†] 马萨诸塞大学阿默斯特分校、[†]美国陆军研究实验室 {jin.Huang, csamplawski, dganesan, marlin}@cs.umass.edu, heesung.kwon.civ@mail.mil

抽象的

近年来，低功耗神经加速器取得了巨大进步，旨在将深度学习分析引入物联网设备。与此同时，低功耗无线电的设计也取得了相当大的进步，以实现从物联网设备到云的高效计算卸载。两者都不是万能药——深度学习模型对于低功耗加速器来说通常太大，而带宽需求对于低功耗无线电来说通常太大。尽管在智能手机级设备的深度学习方面已经做了相当多的工作，但这些方法对于资源受限的小型电池供电物联网设备来说效果不佳。

在本文中，我们通过设计一种连续可调的方法来弥补这一差距，利用本地和远程资源来优化深度学习模型的性能。Clio提出了一种新颖的方法，以适应无线动态的渐进方式在物联网设备和云之间分割机器学习模型。我们证明，这种方法可以与模型压缩和自适应模型分区相结合，创建一个用于物联网云分区的集成系统。我们在 GAP8 低功耗神经加速器上实现了 Clio，提供了每种方法表现最佳的操作机制的详尽特征，并表明 Clio 可以随着资源减少而实现适度的性能下降。

CCS 概念

• 计算机系统组织 → 云计算。

关键词

边缘计算、云计算、计算卸载、深度神经网络

ACM参考格式:

Jin Huang^{*}、Colin Samplawski^{*}、Deepak Ganesan^{*}、Benjamin Marlin^{*}、Heesung Kwon[†]. 2020. CLIO: 实现跨物联网和云的深度学习管道的自动编译。第 26 届移动计算和网络国际年会 (MobiCom '20), 2020 年 9 月 21 日至 25 日, 英国伦敦。ACM, 美国纽约州纽约市, 12 页。https://doi.org/10.1145/3372224.3419215

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

MobiCom '20, September 21–25, 2020, London, United Kingdom

© 2020 Association for Computing Machinery.

ACM ISBN 978-1-4503-7085-1/20/09...\$15.00

https://doi.org/10.1145/3372224.3419215

1 简介

虽然传统的物联网和传感器网络专注于连接简单的传感器（例如恒温器、湿度传感器等），但我们正在看到向更丰富、始终在线的传感器（例如摄像头、麦克风、IMU）的转变，这些传感器近年来变得更加节能[9,18,35,55]。反过来，这些功能正在推动电池供电传感器的更复杂的应用（例如家庭安全、移动健康）。

这些趋势催生了支持低功耗连续分析的相关计算元件的新兴市场。最近发布的几种神经加速器芯片组，如 GreenWaves GAP8 [51]、EtaCompute Tensai [50] 和 Syntiant NDP [52] 功耗低于 100mW，从而使我们能够设想配备神经加速器的 Mote 级设备。

尽管取得了这些进展，低功耗加速器的可用资源与高效执行最先进的深度学习模型所需的资源之间仍然存在很大差距。虽然模型压缩方法已经成功地将大型模型压缩到智能手机和 Raspberry PI 等嵌入式设备中，但低功耗加速器代表了巨大的进步，因为它们的资源比这些嵌入式处理器少两到三个数量级（例如，GAP8 的卷积加速器为 1 GFlops，而 Raspberry Pi 为 24 GFlops，iPhone X 为 350 GFlops）。迄今为止，大多数模型压缩方法的成功都具有相对较小的压缩比（例如，计算和内存占用量小于 2x，而不会显着损失精度[17]），但即使将 ResNet-18、VGG-16 和 MobileNetV2 等最先进的深度学习模型的最小版本安装到物联网设备中，也需要在计算和内存占用量上进行 5-20x 压缩，这超出了能力范围现有方法。因此，现有的物联网机器学习模型优化工作使用了更简单的模型，并依赖于手工优化，而不是最先进的深度学习模型和自动模型压缩技术[26]。无论采用哪种方法，代价都是预测精度大幅降低，因为将模型适应可用资源需要极高的压缩水平。

模型压缩的另一种选择是利用无线通信将数据传输到云进行处理。我们可以不压缩模型，而是使用有损压缩技术压缩原始数据，将其传输到云端，并使用云资源进行预测。这是特别有吸引力的，因为低功耗无线电前端的效率及其占空比效率在过去二十年中稳步提高（例如 BLE、Zigbee 等 [1, 36, 49]）。然而，挑战在于低功率无线电的带宽有限，而且也很敏感

to range and environmental dynamics since they transmit at low power levels. This can be problematic in low bandwidth settings since squeezing data into a narrow wireless link via lossy compression can lead to poor inference performance due to information loss.

These developments raise an interesting question — can we simultaneously leverage both model compression (where the model is squeezed to fit local compute resources) and data compression (where the data is squeezed to fit the narrow wireless link) to maximize inference quality? A growing body of research is exploring the benefits of partitioning deep learning models such that the IoT device executes the initial layers of a convolutional neural network (CNN) and transmits the intermediate results to the cloud where the remaining stages of execution occur (e.g. [21, 24, 40]). However, for most state-of-art models, intermediate results from various layers are often quite large and inefficient to transmit using low-bandwidth IoT radios. This issue is exacerbated by the fact that only the early layers of a deep learning model are viable as partition points given compute and memory constraints on the IoT device. In mobile settings, wireless dynamics also presents a significant issue — existing techniques need to be carefully trained for specific bandwidth settings and cannot adjust on-the-fly to dynamic changes in bandwidth.

The CLIO Approach: In this work, we seek to tackle these problems and answer the important fundamental question of how ultra-low compute and communication components can be simultaneously leveraged to improve inference accuracy. We propose **Clio**¹, a novel model compilation framework that provides a continuum of options for deep learning models to be partitioned across a low-power IoT device and the cloud while gracefully degrading performance in the presence of limited and dynamically varying resources. **Clio** achieves this by combining the idea of partitioning a deep learning model together with progressive transmission of intermediate results — put together, these techniques can deal with both resource constraints and resource dynamics. Importantly, **Clio** integrates progressive transmission with model compression and adaptive partitioning in response to wireless dynamics. By doing so, **Clio** significantly expands the possible endpoints for model compilation for IoT-cloud networks.

One of the key benefits of **Clio** is that it allows model compilation techniques to be used even for stringently resource-constrained IoT devices which may have orders of magnitude less resources than the original model requires. By fully utilizing available bandwidth for remote execution, **Clio** limits the portion of the original model that needs to be compiled to the resource-constrained device thereby reducing the extent of compression that is required. This has significant advantages from a deployment perspective — rather than having to hand-tune a model for deployment, automatic model compilation allows us to deploy large models over resource-limited networks of devices.

We present an exhaustive evaluation of **Clio** across a spectrum of compute architectures [6, 51] radio architectures [1, 3, 5] and datasets [14, 25, 37]. We show that **Clio** can be used to compile state-of-art models like MobileNetV2 [38] and ResNet-18 [16] into resource-constrained IoT settings, and can reduce both end-to-end

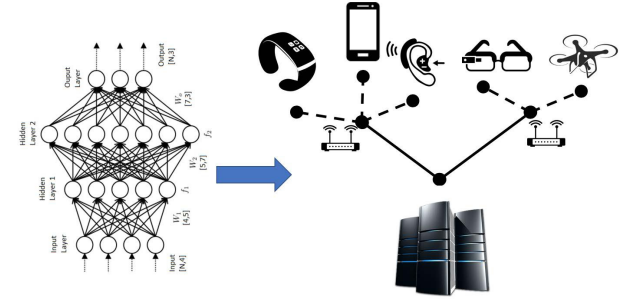


Figure 1: Many IoT devices with rich sensors like cameras are continuously connected to the cloud. We need new methods to execute deep learning models across IoT-Cloud while respecting resource constraints and adapting to wireless variability.

latency and power consumption without a noticeable drop in inference accuracy. We demonstrate significant gains over state-of-art approaches for data compression [32], model compression [17, 33] and partitioned execution [21, 29].

2 CASE FOR CLIO

While there has been significant prior work on deep learning in embedded systems, low-power IoT systems present unique challenges due to their extreme resource constraints.

From a radio perspective, the main challenge is dealing with uncertain wireless conditions. IoT radios operate in a highly duty-cycled manner to save power; however, this comes at the cost of not knowing available bandwidth upon wakeup since the channel may have changed since the previous wake period. The situation is exacerbated by the fact that IoT radios transmit at low power (typically 0dBm) and uplink bandwidth is therefore more sensitive to factors like body blockage, device mobility, and attenuation due to walls. In addition, it is not practical to pre-train and deploy a large number of different models for different available bandwidths in resource constrained settings. Finally, due to loss of synchronization during sleep, there is additional connection latency to start transferring data when an event occurs. This in turn adds to the end-to-end delay for remote computation approaches that rely on the cloud.

From a computation perspective, the key challenge is the lack of sufficient local resources. While low-power accelerators have the architectural building blocks to execute deep learning models quickly, they are still limited by computation ability, memory size and maximum operating frequency. For example, the GAP8 operates at 1 GFlops (with convolutional accelerator), 64KB L1 cache and 512KB L2 cache, and a maximum operating frequency of 175 MHz. This is a challenge since state-of-art deep learning models need two to three orders of magnitude more computational resources than are available.

We now look at canonical approaches for execution of deep learning models at the edge and describe why they are inadequate for low-power IoT scenarios.

¹CLIO stands for **C**loud-**I**oT partitioning

由于它们以低功率水平传输，因此会影响范围和环境动态。这在低带宽设置中可能会出现，因为通过有损压缩将数据压缩到狭窄的无线链路中可能会因信息丢失而导致推理性能不佳。

这些发展提出了一个有趣的问题——我们能否同时利用模型压缩（模型被压缩以适应本地计算资源）和数据压缩（数据被压缩以适应狭窄的无线链路）来最大化推理质量？越来越多的研究正在探索划分深度学习模型的好处，以便物联网设备执行卷积神经网络（CNN）的初始层并将中间结果传输到云，在云中执行其余阶段（例如[21,24,40]）。然而，对于大多数最先进的模型来说，各个层的中间结果通常非常大，并且使用低带宽物联网无线电传输效率低下。由于物联网设备上的计算和内存限制，只有深度学习模型的早期层才可以作为分区点，这一事实加剧了这个问题。在移动设置中，无线动态也提出了一个重大问题——现有技术需要针对特定带宽设置进行仔细训练，并且无法即时调整以适应带宽的动态变化。

CLIO 方法：在这项工作中，我们寻求解决这些问题并回答如何同时利用超低计算和通信组件来提高推理精度的重要基本问题。我们提出了 Clio1，这是一种新颖的模型编译框架，它为深度学习模型提供了连续的选项，可以在低功耗物联网设备和云之间进行分区，同时在资源有限且动态变化的情况下适度降低性能。Clio 通过将深度学习模型划分的思想与中间结果的渐进式传输相结合来实现这一目标 - 总而言之，这些技术可以处理资源约束和资源动态。重要的是，Clio 将渐进式传输与模型压缩和自适应分区相结合，以响应无线动态。通过这样做，Clio 显着扩展了物联网云网络模型编译的可能端点。

Clio 的主要优点之一是它允许使用模型编译技术，甚至可以用于资源严格受限的物联网设备，这些设备的资源可能比原始模型所需的资源少几个数量级。通过充分利用远程执行的可用带宽，Clio 限制了原始模型中需要编译到资源受限设备的部分，从而减少了所需的压缩程度。从部署的角度来看，这具有显著的优势——自动模型编译允许我们在资源有限的设备网络上部署大型模型，而不必手动调整模型进行部署。

我们对 Clio 跨一系列计算架构 [6, 51]、无线架构 [1, 3, 5] 和数据集 [14, 25, 37] 进行了详尽的评估。我们证明，Clio 可用于将 MobileNetV2 [38] 和 ResNet-18 [16] 等最先进的模型编译到资源受限的物联网设置中，并且可以减少端到端的模型

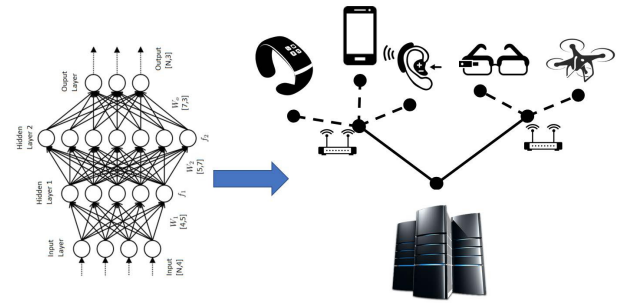


图 1：许多具有丰富传感器（例如摄像头）的物联网设备持续连接到云端。我们需要新的方法来跨物联网云执行深度学习模型，同时尊重资源限制并适应无线变化。

延迟和功耗，而推理精度没有明显下降。我们展示了数据压缩 [32]、模型压缩 [17, 33] 和分区执行 [21, 29] 方面最先进方法的显着优势。

2 克里奥 (CLIO) 案例

尽管之前在嵌入式系统中的深度学习方面已经开展了大量工作，但低功耗物联网系统由于其极端的资源限制而面临着独特的挑战。

从无线电角度来看，主要挑战是处理不确定的无线条件。物联网无线电以高负载循环方式运行以节省电力；然而，这是以不知道唤醒时的可用带宽为代价的，因为自上一个唤醒周期以来信道可能已经改变。由于物联网无线电以低功率（通常为 0dBm）传输，因此上行链路带宽对身体遮挡、设备移动性和墙壁衰减等因素更加敏感，这一事实加剧了这种情况。此外，在资源受限的环境中针对不同的可用带宽预训练和部署大量不同的模型是不切实际的。最后，由于睡眠期间失去同步，因此在事件发生时开始传输数据会存在额外的连接延迟。这反过来又增加了依赖云的远程计算方法的端到端延迟。

从计算的角度来看，关键的挑战是缺乏足够的本地资源。虽然低功耗加速器具有快速执行深度学习模型的架构构建块，但它们仍然受到计算能力、内存大小和最大运行频率的限制。例如，GAP8 的运行速度为 1 GFlops（带卷积加速器）、64KB L1 缓存和 512KB L2 缓存，最大运行频率为 175 MHz。这是一个挑战，因为最先进的深度学习模型需要比可用资源多两到三个数量级的计算资源。

我们现在研究在边缘执行深度学习模型的规范方法，并描述为什么它们不适用于低功耗物联网场景。

¹CLIO stands for Cloud-IoT partitioning

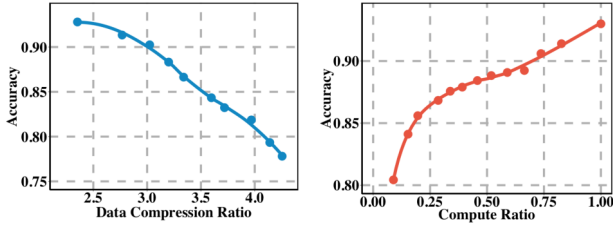


Figure 2: (a) Remote execution is bottlenecked by bandwidth availability. High data compression ratios lead to poor accuracy. (b) Local execution is bottlenecked by compute/memory availability. At low compute, model compression perform particularly poorly.

Data compression vs model compression: Two common methods for dealing with limited resources are data compression (i.e., we squeeze the data volume by transmitting a lossy version of the data for remote execution) and model compression (i.e., we squeeze larger models to execute locally within desired latency bounds). However, in severely resource-constrained scenarios, both approaches can have low prediction performance.

We illustrate this tradeoff in Figure 2 — the figure shows the effect of reducing data transmission size and reducing model size on accuracy when executing the MobileNetV2 model on CIFAR-10 images. On the left is the effect of data compression using JPEG, a popular lossy compression method for images. The curve corresponds to accuracy when executing the MobileNetV2 model on images compressed at different JPEG compression qualities. We train a separate classification model for each JPEG quality to show best-case performance given images of a particular JPEG quality. On the right is the effect of a particular model compression approach based on scaling down the layer widths of the MobileNetV2 model, which reduces the required computation.²

The figure shows that accuracy degrades rather quickly once the data or model are reduced in size. Even at relatively low compression levels, the accuracy drops below 90% and steadily reduces thereafter. We see a similar trend for ImageNet.

We note that replacing model compression with hand-crafted models that are specifically designed to fit into resource-constrained MCUs does not resolve the problem. For example, the CMSIS-NN is a convolutional neural network specifically optimized for the ARM STM32 F7 [27], but the model only achieves around 80% accuracy for the CIFAR-10 dataset compared to the 93% accuracy that the full MobileNetV2 model achieves. Hence, such hand-crafted methods can also come with significant performance degradation.

Model partitioning: Another approach that has become popular is edge-cloud model partitioning to take advantage of both remote and local compute resources to optimize performance [15, 21, 29, 41]. While partitioning is appealing, existing methods assume knowledge of bandwidth to determine how to partition the model and do

not directly tackle the issues of bandwidth uncertainty and variability that we described earlier. Also, resource constraints and high dynamic range of bandwidth also make it infeasible to store a large number of models optimized for different bandwidths.

The Clio approach: **Clio** is a model compilation technique that allows us to execute large state-of-art models over resource-constrained IoT networks while gracefully degrading performance in the face of uncertain wireless bandwidth. At the core of **Clio** is a joint IoT-cloud optimization technique for progressively transmitting intermediate results from a partitioned model in-order to deal with bandwidth variations. In addition to introducing a new technique for enhancing the operation of partitioned models, **Clio** also presents an integrated solution that combines various pieces of the puzzle — model compression, model partitioning and progressive transmission — to create an integrated system for partitioned execution under dynamic bandwidth settings. Thus, our work is holistic and leverages multiple methods to provide the best overall performance. We describe our proposed solution in more detail in the following section.

3 CLIO DESIGN

We start with an overview of the **Clio** run-time and then describe the core techniques involved.

3.1 Overview of Clio run-time

The **Clio** runtime operates differently in response to short-term and long-term bandwidth dynamics. There are two cases where rapid adaptation to bandwidth is required. The first is when channel conditions are unknown when a node wakes up after operating in deep sleep (i.e. during duty-cycled operation). The second is when link bandwidth varies unpredictably due to mobility. **Clio** adapts on-the-fly to these conditions by operating in a progressive manner.

When bandwidth changes persist for longer periods, then **Clio** adapts the partition point so that it can operate at the best partition for the current conditions. For example, say the model is partitioned at layer 5 and progressive slicing can allow it to deal with bandwidth swings from 250 kbps to 1 Mbps. If the average bandwidth now drops to 100 kbps, then switching to a different partition point is needed in-order to further shrink the size of intermediate results and better cope with the lower available bandwidth.

3.2 Progressive slicing

Preliminaries: A deep learning model M can be represented by a directed acyclic computation graph $G = (V, E)$ consisting of a set of layers V and a set of directed edges E . The **partition point** of the graph is a cut $C \subset E$ in the computation graph G that separate the model M into two parts.

To simplify this optimization problem, we restrict the cut to occur between two successive layers in the model. We assume a model with K layers and a cut occurs between L_k and L_{k+1} , $k \in \{1, \dots, K-1\}$. Thus, in the layered case, the choice of the cut $C \in C$ determines the trade-off between end-to-end latency and total energy consumption. At this point, accuracy is unchanged as we are simply distributing a computation graph across the edge device and the cloud.

²We note that there are many approaches to model compression including weight quantization, weight sparsification, compact filters, and so on. The example we provide is one illustration of the general trend.

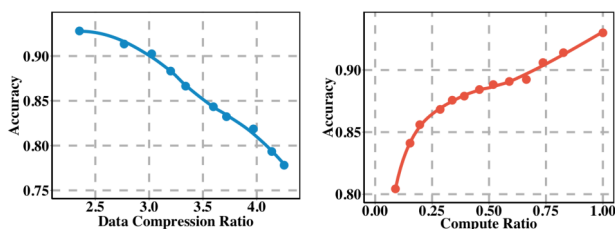


图 2: (a) 远程执行受到带宽可用性的瓶颈。高数据压缩率会导致准确性较差。(b) 本地执行受到计算/内存可用性的瓶颈。在低计算量下, 模型压缩的性能特别差。

数据压缩与模型压缩: 处理有限资源的两种常见方法是数据压缩 (即, 我们通过传输有损版本的数据进行远程执行来压缩数据量) 和模型压缩 (即, 我们压缩较大的模型以在所需的延迟范围内本地执行)。然而, 在资源严重受限的情况下, 这两种方法的预测性能都较低。

我们在图 2 中说明了这种权衡 - 该图显示了在 CIFAR-10 图像上执行 MobileNetV2 模型时减少数据传输大小和减少模型大小对准确性的影响。左边是使用 JPEG (一种流行的图像有损压缩方法) 进行数据压缩的效果。该曲线对应于对以不同 JPEG 压缩质量压缩的图像执行 MobileNetV2 模型时的准确度。我们为每种 JPEG 质量训练一个单独的分类模型, 以显示给定特定 JPEG 质量图像的最佳情况性能。右侧是基于缩小 MobileNetV2 模型层宽度的特定模型压缩方法的效果, 这减少了所需的计算量。²

该图显示, 一旦数据或模型尺寸减小, 准确性就会很快下降。即使在相对较低的压缩水平下, 准确度也会降至 90% 以下, 并在此后稳步下降。我们在 ImageNet 中看到了类似的趋势。

我们注意到, 用专门设计用于资源受限的 MCU 的手工模型替换模型压缩并不能解决问题。例如, CMSIS-NN 是专门针对 ARM STM32 F7 [27] 进行优化的卷积神经网络, 但该模型对于 CIFAR-10 数据集仅实现了约 80% 的准确率, 而完整的 MobileNetV2 模型可实现 93% 的准确率。因此, 这种手工方法也会导致性能显著下降。

模型分区: 另一种流行的方法是边缘云模型分区, 以利用远程和本地计算资源来优化性能[15,21,29,41]。虽然分区很有吸引力, 但现有方法假设带宽知识来确定如何分区模型并执行

不能直接解决我们之前描述的带宽不确定性和可变性问题。此外, 资源限制和带宽的高动态范围也使得存储大量针对不同带宽优化的模型变得不可行。

Clio 方法: Clio 是一种模型编译技术, 使我们能够在资源有限的物联网网络上执行大型最先进的模型, 同时在面对不确定的无线带宽时优雅地降低性能。Clio 的核心是物联网-云联合优化技术, 用于逐步传输分区模型的中间结果, 以应对带宽变化。除了引入增强分区模型操作的新技术之外, Clio 还提出了一种集成解决方案, 该解决方案结合了模型压缩、模型分区和渐进式传输等各个难题, 以创建一个在动态带宽设置下进行分区执行的集成系统。因此, 我们的工作整体性的, 并利用多种方法来提供最佳的整体性能。我们将在下一节中更详细地描述我们提出的解决方案。

3 克里奥设计

我们首先概述 Clio 运行时, 然后描述所涉及的核心技术。

3.1 Clio运行时概述

Clio 运行时的运行方式不同, 以响应短期和长期带宽动态。有两种情况需要快速适应带宽。第一种情况是, 当节点在深度睡眠模式下运行 (即在占空比运行期间) 后醒来时, 信道条件未知。第二种情况是链路带宽由于移动性而发生不可预测的变化。Clio 通过渐进式操作来即时适应这些条件。

当带宽变化持续较长时间时, Clio 会调整分区点, 以便它可以在当前条件下的最佳分区上运行。例如, 假设模型在第 5 层进行分区, 渐进式切片可以使其能够处理从 250 kbps 到 1 Mbps 的带宽波动。如果平均带宽现在下降到 100 kbps, 则需要切换到不同的分区点, 以便进一步缩小中间结果的大小并更好地应对较低的可用带宽。

3.2 渐进切片

预备知识: 深度学习模型 M 可以用有向无环计算图 $G = (V, E)$ 表示, 该图由一组层 V 和一组有向边 E 组成。该图的分割点是计算图 G 中的一条割线 $C \subset E$, 它将模型 M 分为两部分。

为了简化这个优化问题, 我们限制切割发生在模型中的两个连续层之间。我们假设模型具有 K 层, 并且在 L_k 和 L_{k+1} 、 $k \in \{1, \dots, K-1\}$ 之间发生剪切。因此, 在分层情况下, 切割 $C \in C$ 的选择决定了端到端延迟和总能耗之间的权衡。此时, 准确性没有改变, 因为我们只是将计算图分布在边缘设备和云上。

²We note that there are many approaches to model compression including weight quantization, weight sparsification, compact filters, and so on. The example we provide is one illustration of the general trend.

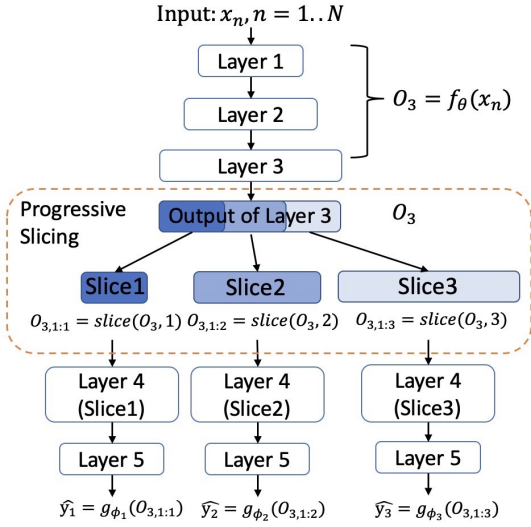


Figure 3: Example of progressive slicing.

Learning Ordered Hidden Representations: Clio continuously adapts the size of the intermediate activations transmitted in a progressive manner to maximize best inference under the given bandwidth conditions. We achieve this via a novel training procedure that learns a single model that can continually adapt the size of the hidden representation that it transmits from layer L_k on the edge device to layer L_{k+1} on the cloud. We learn a separate model for the cloud for each hidden representation size. These models receive and decode the transmitted representation from layer L_k of each size, producing a final output.

Let $O_k = f_\theta(\mathbf{x})$ be a function implementing the computation graph for a model M up to the output of layer k and assume that this layer includes W_k units. Now, define the slicing function $O_{1:i} = \text{slice}(i, O)$ to map the input vector O to the sub-vector of O consisting of its first i channels, $O_{1:i}$. Define $y = g_{\phi_i}(O_{1:i,k})$ to be a function implementing the computation graph for a model M that receives as input only the first i hidden unit values from layer k and produces the final output of the network. Note that this function has a separate set of parameters for each value of i .

The model transmits as many hidden units as possible from layer k to layer $k+1$ across the wireless link given the latency constraint and available bandwidth. The final prediction output by the model is then given by: $y = g_{\phi_i}(\text{slice}(i, f_\theta(\mathbf{x})))$

The key to the success of this approach is to train the model using a loss function that takes into account a distribution over available bandwidths and thus over the representation sizes that can be transmitted. Let π_i be the probability that there is sufficient bandwidth available to send $\text{slice}(i, f_\theta(\mathbf{x}))$. Given a data set $\mathcal{D} = \{(\mathbf{x}_n, y_n), n = 1 : N\}$ and a prediction loss function $\ell(y, y')$, we define the loss based on the intermediate representation $\text{slice}(i, f_\theta(\mathbf{x}))$ by $\mathcal{L}_i(\mathcal{D}; \theta, \phi_i)$. The overall loss that we optimize is the expected value of $\mathcal{L}_i(\mathcal{D}; \theta, \phi_i)$ where the expectation is taken over the probability π_i that we can send the representation $\text{slice}(i, f_\theta(\mathbf{x}))$. The model thus learns parameters that result in an ordered hidden representation with the property that it minimizes

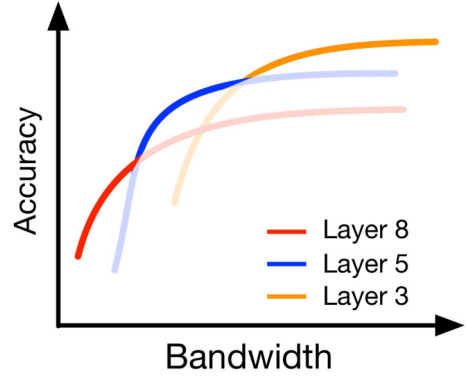


Figure 4: For each choice of partition point, progressive slicing offers a range of accuracy–bandwidth tradeoffs. The envelope (in bold) is the best operating regime for given bandwidth.

expected error under the progressive slicing operation for the given distribution over representation sizes π . The details are provided below.

$$\theta^*, \phi_{1:W_k}^* = \arg \min_{\theta, \phi_{1:W_k}} \mathbb{E}_\pi [\mathcal{L}_i(\mathcal{D}; \theta, \phi_i)] \quad (1)$$

$$\mathcal{L}_i(\mathcal{D}; \theta, \phi_i) = \sum_{n=1}^N \ell(y_n, g_{\phi_i}(\text{slice}(i, f_\theta(\mathbf{x}_n)))) \quad (2)$$

Example: We illustrate progressive slicing with the example shown in Figure 3. Here, the IoT device executes the initial layers until layer 3, and the rest of the pipeline is executed at the cloud. In this example, the output of layer 3 is partitioned into three slices, and depending on the bandwidth, a different number of slices are transmitted to the cloud. There are three corresponding versions of layers on the cloud to handle the three possible slices as input. During inference time, IoT devices will execute layers 1 – 3 locally and pass the intermediate result of layer 3 to radio for transmission. Due to network dynamics, the radio will send as much as possible in the available time and the cloud will activate the corresponding path for the received progressively sliced intermediate results.

3.3 Adaptive partitioning

So far, we have assumed that we know the best partition point. But the choice of partition point impacts performance since changing it affects both the latency for computation, as well as the size of the activations. While progressive slicing does provide the ability to deal with variations in the data transmission size, a single partition point may not provide sufficient dynamic range across bandwidth conditions.

Figure 4 illustrates this interplay between partitioning and progressive transmission with an example. For each partition point, progressive transmission is a curve that offers different accuracy – bandwidth tradeoffs. Thus, the best operating point given some specific bandwidth is the (partition, slice) tuple that has the highest accuracy at that bandwidth.

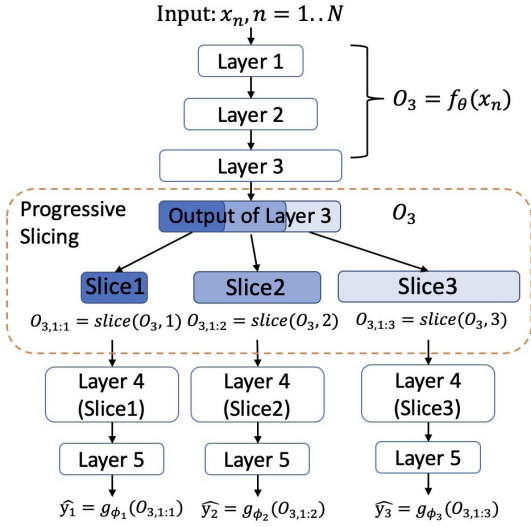


图 3: 渐进式切片示例。

学习有序隐藏表示: Clio 不断调整以渐进方式传输的中间激活的大小, 以在给定带宽条件下最大化最佳推理。我们通过一种新颖的训练程序来实现这一目标, 该程序学习一个模型, 该模型可以不断调整从边缘设备上的 L_k 层传输到云上的 L_{k+1} 层的隐藏表示的大小。我们为每个隐藏表示大小学习一个单独的云模型。这些模型接收并解码来自每个尺寸的 L_k 层传输的表示, 产生最终输出。

令 $O_k = f_\theta(x)$ 为实现模型 M 直至层 k 输出的计算图的函数, 并假设该层包含 W_k 单元。现在, 定义切片函数 $O1:i = \text{slice}(i, O)$ 将输入向量 O 映射到由其第一个 i 通道 $O1:i$ 组成的 O 的子向量。将 $y = g_{\phi_i}(O1:i, k)$ 定义为实现模型 M 计算图的函数, 该模型仅接收来自层 k 的第一个 i 隐藏单元值作为输入, 并生成网络的最终输出。请注意, 此函数对于 i 的每个值都有一组单独的参数。

在给定延迟约束和可用带宽的情况下, 模型通过无线链路从第 k 层到第 $k+1$ 层传输尽可能多的隐藏单元。模型的最终预测输出由下式给出: $y = g_{\phi_i}(\text{slice}(i, f_\theta(x)))$

这种方法成功的关键是使用损失函数来训练模型, 该函数考虑了可用带宽上的分布, 从而考虑了可以传输的表示大小的分布。令 π_i 为有足够带宽可用于发送切片 $(i, f_\theta(x))$ 的概率。给定数据集 $\mathcal{D} = \{(x_n, y_n), n = 1 : N\}$ 和预测损失函数 $\ell(y, y')$, 我们通过 $\mathcal{L}_i(\mathcal{D})$ 定义基于中间表示切片 $(i, f_\theta(x))$ 的损失; θ, ϕ_i 。我们优化的总体损失是 $\mathcal{L}_i(\mathcal{D})$ 的期望值; θ, ϕ_i , 其中期望取自我们可以发送表示切片 $(i, f_\theta(x))$ 的概率 π_i 。因此, 该模型学习参数, 这些参数导致有序隐藏表示及其最小化的属性

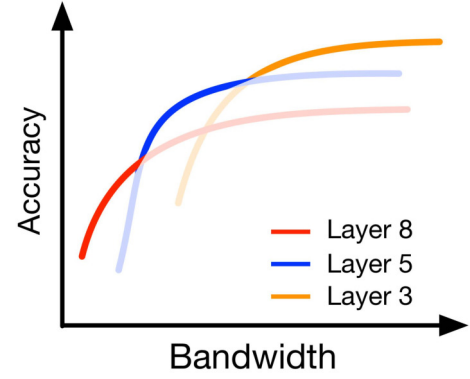


图 4: 对于每种划分点的选择, 渐进式切片提供了一系列精度与带宽的权衡。包络线 (粗体) 是给定带宽的最佳运行状态。

对于表示大小 π 的给定分布, 渐进切片操作下的预期误差。详细信息如下。

$$\theta^*, \phi_{1:W_k}^* = \arg \min_{\theta, \phi_{1:W_k}} \mathbb{E}_\pi [\mathcal{L}_i(\mathcal{D}; \theta, \phi_i)] \quad (1)$$

$$\mathcal{L}_i(\mathcal{D}; \theta, \phi_i) = \sum_{n=1}^N \ell(y_n, g_{\phi_i}(\text{slice}(i, f_\theta(x_n)))) \quad (2)$$

示例: 我们用图 3 中所示的示例来说明渐进式切片。在这里, 物联网设备执行初始层直到第 3 层, 管道的其余部分在云端执行。在此示例中, 第 3 层的输出被划分为三个切片, 并且根据带宽, 将不同数量的切片传输到云端。云上有三个相应版本的层来处理三个可能的切片作为输入。在推理期间, 物联网设备将在本地执行第 1 层 - 3 层, 并将第 3 层的中间结果传递到无线电进行传输。由于网络动态, 无线电将在可用时间内发送尽可能多的数据, 云将为接收到的逐步切片的中间结果激活相应的路径。

3.3 自适应分区

到目前为止, 我们假设我们知道最好的划分点。但是分区点的选择会影响性能, 因为更改它会影响计算延迟以及激活的大小。虽然渐进切片确实提供了处理数据传输大小变化的能力, 但单个划分点可能无法在带宽条件下提供足够的动态范围。

图 4 通过一个示例说明了分区和渐进式传输之间的相互作用。对于每个分区点, 渐进传输是一条提供不同精度-带宽权衡的曲线。因此, 给定某些特定带宽的最佳操作点是在该带宽下具有最高准确度的 $(\text{partition}, \text{slice})$ 元组。

Thus, we build a performance profile similar to that shown in Figure 4 i.e. a set of performance curves offered by different progressive slices across different partition points. These profiles can be calculated using platform-level benchmarking as we do on the GAP8 device in §4.2. Once we have such a profile, we can select the optimal partition point and progressive slice for a given bandwidth. We note that building such a profile may be expensive in terms of training time due to the number of possible partition points and progressive slices; we discuss how to optimize training time in §3.4.

Incorporating model compression: Augmenting progressive slicing with model compression can improve performance, particularly for computation of the IoT-portion of the model which is slower due to resource constraints at the IoT node. There are several approaches used for model compression onto embedded devices [10] including weight quantization, weight sparsification, compact filters and others. We apply these model compression methods at different stages of our pipeline — pruning happens prior to progressive slicing whereas weight quantization and sparsification happens after the progressively sliced model has been trained. This allows us to easily integrate progressive slicing with state-of-the-art model compression techniques (e.g. our implementation integrates with AutoML [17] and Meta-pruning [33]).

3.4 Reducing training time

So far, we assume that we train across all possible slices of the intermediate layer (for progressive transmission) and across all possible layers as potential partition points. Clearly, this can incur significant training time. In this section, we describe several optimizations to reduce training time.

Let $C(\mathbf{L}_{1:k})$ be the computation cost of the model $\mathbf{M}$ of layers $\mathbf{L}_{1:k}$ and k be the partition points. For the training of progressive slicing, the early layers $\mathbf{L}_{1:k}$ will be shared (since they are unaffected by slicing), and the set of layers $\mathbf{L}_{k+1:K}$ will be different for each slice. Thus, the computation cost of the model will be $C(\mathbf{M}) = C(\mathbf{L}_{1:k}) + \text{\#slices} \times C(\mathbf{L}_{k+1:K})$. Therefore, training cost is affected by two parameters — the number of slices and the number of partition points (i.e. k). We now look at how to optimize these two aspects.

Reducing training time for progressive slicing: There are two optimization that we use in our training to greatly reduce training complexity for training across progressive slices. First, we limit the number of different hidden representation sizes that are considered by setting some values of π_i to 0. For example, in our work we consider only sending a number of hidden units that is a multiple or power of a given integer. This approach can be used to restrict the number of different parameter sets required in the cloud to a more manageable number for large models. Second, we tie the parameters starting at layer $k + 2$ so that we are only expanding the required parameters at the first layer on the cloud. For example, in Figure 3 we see that the cloud uses separate pipelines depending on which slice is transmitted. Instead, the cloud can only have different versions of Layer 4 to deal with the different number of intermediate activations transmitted and use the same Layer 5 across all these Layer 4 options. Our implementation uses both of these optimizations to accelerate training.

Reducing potential partition points: To reduce training across partition points, we use a bandwidth coverage heuristic to choose

a few promising partition points to reduce training time. For each layer where we can partition the model, progressive slicing can provide some tolerance to bandwidth swings by changing how many slices are transmitted. So, we use a greedy algorithm to select only layers that can extend the bandwidth range to which the model can adapt. We now describe this process in more detail.

In-order to estimate how much bandwidth range can be supported via progressive slicing on a layer, we first need to know how much time is available for communication. Let $C_{iot}(\cdot)$ and $C_{cloud}(\cdot)$ denote the computation latency at the IoT device and cloud respectively. These functions can be estimated for each layer through one-time offline profiling.

For a model partitioned at layer K , $\mathbf{L}_{1:k}$ are executed at the IoT device and $\mathbf{L}_{k+1:K}$ at the cloud. The total latency for computation, ϵ_k^C , is $C_{iot}(\mathbf{L}_{1:k}) + C_{cloud}(\mathbf{L}_{k+1:K})$; total energy for computation will be ϵ_k^C , is $C_{iot}(\mathbf{L}_{1:k})$. Given an end-to-end latency target or total energy consumption of ϵ_{target} , we can now only use $\epsilon_{target} - \epsilon_k^C$ for communication to transmit intermediate results. If $\mathbf{O}_k$ denotes the different sizes of the progressively sliced intermediate results from layer k and $R(\cdot)$ denotes the function that maps the transmission sizes back into the required bandwidths based on target latency, we can calculate the bandwidth range $\mathbf{B}_k$ that progressive slicing at layer k can dynamically adjust to as:

$$\mathbf{B}_k = R(\mathbf{O}_k, \epsilon_{target} - \epsilon_k^C) \quad (3)$$

Given this range, we just select the most promising partitions as the minimum set of intervals that cover the bandwidth range of interest (e.g. from 50kbps to 2Mbps for Bluetooth). This is a simple interval coverage problem that can be solved using a greedy algorithm [12].

4 IMPLEMENTATION

We instantiate **Clio** for several state-of-the-art neural network models and IoT accelerators and processors, and evaluate on different datasets. We describe the implementation details below.

4.1 Compiling Clio to Models

To prove the generalization of our approach, we apply **Clio** to several different state-of-the-art neural network models including **MobileNetV2** [38], **VGG** [42] and **ResNet** [16]. We partition after any layer or blocks (residual blocks in **ResNet** or bottleneck structures in **MobileNetV2**) and do not partition within each block structure.

Skip connections: MobileNetV2 uses skip connections to connect the beginning and end of a convolutional block; by doing so, the network has the opportunity of accessing earlier activations that were not modified in the convolutional block. Skip connections usually exist when the input and output have the same size. Since **Clio** progressively slices the intermediate results into different sizes during training, we change the skip connection from an addition operation to a 1×1 convolutional operation so that we can deal with different sizes of input and output.

因此，我们构建了一个类似于图 4 所示的性能概况，即由跨不同分区点的不同渐进切片提供的一组性能曲线。这些配置文件可以使用平台级基准测试来计算，就像我们在第 4.2 节中在 GAP8 设备上所做的那样。一旦我们有了这样的配置文件，我们就可以为给定的带宽选择最佳的划分点和渐进切片。我们注意到，由于可能的分区点和渐进切片的数量，构建这样的配置文件在训练时间方面可能会很昂贵；我们在第 3.4 节中讨论如何优化训练时间。

合并模型压缩：通过模型压缩增强渐进式切片可以提高性能，特别是对于模型的物联网部分的计算，由于物联网节点的资源限制，该部分的计算速度较慢。有多种方法可用于嵌入式设备上的模型压缩[10]，包括权重量化、权重稀疏、紧凑滤波器等。我们在管道的不同阶段应用这些模型压缩方法——修剪发生在渐进切片之前，而权重量化和稀疏化发生在渐进切片模型训练之后。这使我们能够轻松地将渐进式切片与最先进的模型压缩技术集成（例如，我们的实现与 AutoML [17] 和元剪枝 [33] 集成）。

3.4 减少训练时间

到目前为止，我们假设我们训练了中间层的所有可能的切片（用于渐进传输）以及所有可能的层作为潜在的分点。显然，这会花费大量的培训时间。在本节中，我们描述了几种减少训练时间的优化。

令 $C(L1:k)$ 为模型 M 层 $L1:k$ 的计算成本， k 为划分点。对于渐进切片的训练，早期层 $L1:k$ 将被共享（因为它们不受切片的影响），并且每个切片的层集 $Lk+1:K$ 将不同。因此，模型的计算成本将为 $C(M) = C(L1:k) + \#slices \times C(Lk+1:K)$ 。因此，训练成本受两个参数影响——切片数量和分点数量（即 k ）。我们现在看看如何优化这两方面。

减少渐进切片的训练时间：我们在训练中使用了两种优化来大大降低跨渐进切片训练的训练复杂性。首先，我们通过将 π_i 的某些值设置为 0 来限制所考虑的不同隐藏表示大小的数量。例如，在我们的工作中，我们考虑仅发送给定整数的倍数或幂的隐藏单元的数量。这种方法可用于将云中所需的不同参数集的数量限制为大型模型更易于管理的数量。其次，我们从第 $k+2$ 层开始绑定参数，以便我们仅在云上的第一层扩展所需的参数。例如，在图 3 中，我们看到云根据传输的切片使用单独的管道。相反，云只能使用不同版本的第 4 层来处理传输的不同数量的中间激活，并在所有这些第 4 层选项中使用相同的第 5 层。我们的实现使用这两种优化来加速训练。

减少潜在的分点：为了减少跨分点的训练，我们使用带宽覆盖启发式来选择

一些有前途的分点可以减少训练时间。对于我们可以划分模型的每一层，渐进切片可以通过改变传输的切片数量来提供一定程度的带宽波动容忍。因此，我们使用贪心算法来仅选择可以扩展模型可以适应的带宽范围的层。我们现在更详细地描述这个过程。

为了估计通过层上的渐进式切片可以支持多少带宽范围，我们首先需要知道有多少时间可用于通信。让 $C_{iot}(\cdot)$ 和 $C_{cloud}(\cdot)$ 分别表示物联网设备和云的计算延迟。可以通过一次性离线分析来估计每一层的这些功能。

对于在 K 层划分的模型， $L1:k$ 在 IoT 设备上执行， $Lk+1:K$ 在云端执行。计算的总延迟 ϵ_k^C 为 $C_{iot}(L1:k) + C_{cloud}(Lk+1:K)$ ；计算的总能量将是 ϵ_k^C ，即 $C_{iot}(L1:k)$ 。给定端到端延迟目标或总能耗为 ϵ_{target} ，我们现在只能使用 $\epsilon_{target} - \epsilon_k^C$ 进行通信来传输中间结果。如果 O_k 表示来自第 k 层的渐进切片中间结果的不同大小，而 $R(\cdot)$ 表示根据目标延迟将传输大小映射回所需带宽的函数，我们可以计算出第 k 层渐进切片可以动态调整的带宽范围 B_k ：

$$B_k = R(O_k, \epsilon_{target} - \epsilon_k^C) \quad (3)$$

给定这个范围，我们只选择最有希望的分点作为覆盖感兴趣的带宽范围的最小间隔集（例如，蓝牙从 50kbps 到 2Mbps）。这是一个简单的区间覆盖问题，可以使用贪心算法来解决[12]。

4 实施

我们为几种最先进的神经网络模型以及物联网加速器和处理器实例化了 Clio，并在不同的数据集上进行了评估。我们在下面描述实施细节。

4.1 将 Clio 编译为模型

为了证明我们方法的泛化性，我们将 Clio 应用于几种不同的最先进的神经网络模型，包括 MobileNetV2 [38]、VGG [42] 和 ResNet [16]。我们在任何层或块（ResNet 中的剩余块或 MobileNetV2 中的瓶颈结构）之后进行分区，并且不在每个块结构内进行分区。

跳跃连接：MobileNetV2 使用跳跃连接来连接卷积块的开头和结尾；通过这样做，网络有机会访问卷积块中未修改的早期激活。当输入和输出具有相同大小时，通常存在跳过连接。由于 Clio 在训练过程中逐步将中间结果切片为不同大小，因此我们将跳跃连接从加法操作更改为 1×1 卷积操作，以便我们可以处理不同大小的输入和输出。

Model compression: As mentioned in §3.4, **Clio** can be incorporated with state-of-the-art compression techniques. In our implementation, we apply AutoML[17] and Meta Pruning [33] to MobileNetV2. We also use width multiplier as a baseline to compress the MobileNetV2 model.

Reducing training time: In order to reduce training time when compiling **Clio** to MobileNetV2, we choose every second channel number for early partition points and every fourth channel number for later partition points for progressive slicing. For example, when we use a target latency of 300ms in our implementation, partition points after layer 3, 5 or 8 of MobileNetV2 are chosen based on our heuristic because they can extend our bandwidth range to 195 kbps, 175 kbps and 98 kbps respectively.

4.2 Compiling Models to IoT Devices

Clio compiles models into two sub-models — a sub-model that is locally executed on IoT devices and a sub-model that is remotely executed on cloud servers. We look at two IoT processors. The **GAP8** is an ultra-low power neural accelerator that has the Hardware Convolutional Engine (HWCE) that can accelerate the convolutional operations [51]. The **STM32F7** is a popular ARM7-based IoT processor without a convolutional accelerator [6]. Our evaluation of MobileNetV2 on the GAP8 accelerator is based on a full end-to-end implementation (described in more detail below). For the ARM7 processor, we estimate the computational cost based on our implementation on the GAP8 but using ARM7 frequency and Risc-V cycles per instructions. In the ARM7, operations such as multiplication, addition and comparison are executed in parallel in the MCU.

GAP8 implementation: The GAP8 platform only has basic support for shallow models and wireless communication, and we were using both larger models as well as integration of IoT and cloud models. We implemented the necessary drivers and optimized them to achieve continuous operation over BLE. We also wrote software to take the model from PyTorch/TFLite and split it/load it onto the GAP8, which dealt with changes in format and optimized it to make it fit in memory.

We use TFLite [53] files for model deployment on GAP8. When converting a model into a TFLite file, some of the operations such as batch normalization are fused for deployment. As GAP8 only supports fixed-point arithmetic operations, all the weights and inputs are transformed into 8-bit Q-format fixed-point numbers during quantization. Meanwhile, the intermediate results are also stored and transmitted as 8-bit fixed point numbers.

There are two kinds of convolutions in MobileNetV2 — the first is basic convolution and the second is expanded convolution, which in-turn consists of three basic convolutions — expansion, depthwise and projection convolutions. Even though the GAP8 has a hardware convolution engine to speed up convolution operations, some operations are not supported due to the hardware design and are executed on the computing cores in parallel. Also, operations such as ReLU6 and padding are fused and executed on the computing cores in parallel.

On the hardware side, we integrated the GAP8 board with a HIMAX camera, LCD screen, and a radio shield which connects via UART interface (shown in Figure 5). The integrated board captures

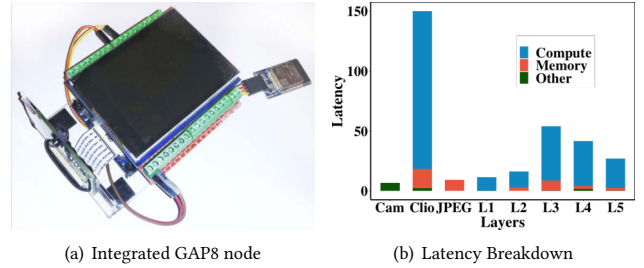


Figure 5: (a) Integrated GAP8 node with Greyscale HIMAX HM01B0 camera, BLE shield, and LCD display, (b) latency specs based on empirical measurement.

an image from camera, shows it on the LCD screen, executes the early layers of the model and then transmits the intermediate results via radios.

Cloud Execution: Execution at the cloud is considerably faster than at the IoT device. For our benchmarks, we use a Xeon Silver 4116 2.10 GHz with 32 GB DDR4 2400 MHz and GTX 1080 Ti GPU.

Demo of real-time operation: We have integrated the above pieces to support continuous live operation. We made a full video demo of our system (see code[2]) — to make the demo, we had to re-train and optimize the model for the greyscale format coming from the GAP8 board’s HIMAX camera which was different from the ImageNet/CIFAR-10 data that we used for our results.

Benchmarking MobileNetV2 on GAP8: We profiled the GAP8 to obtain latency and energy benchmarks for different computational blocks.

For latency profiling, we use the hardware counters to evaluate total cycles for computation, memory copying and other operations. We transform the total cycles to latency by dividing by the GAP8 clock frequency setting. Figure 5 shows the breakdown of latency consumed by the execution of our GAP8 implementation of the MobileNetV2 early layers. Based on the latency breakdown, we estimate the equivalent compute ability of the GAP8 when executing MobileNetV2 to be approximately 525 MMAC/s.

For wireless, we found that the maximum speed at which we could transfer data from the GAP8 to the BLE shield is 250kbps (due to UART speed limitations). We use these numbers in our evaluation when evaluating latency and energy.

5 EVALUATION

We start by describing the datasets we use and schemes we compare against before evaluating **Clio** against these methods.

5.1 Datasets

The two main datasets that we will use for evaluation are based on **CIFAR-10** and **ImageNet** — these datasets have different image sizes and demonstrate the performance of our methods under different workloads. The **CIFAR-10** [25] dataset consists of 60000 32×32 color images in 10 classes, with 6000 images per class. There are 50000 training images and 10000 test images. The **ImageNet** [14] dataset has the 224×224 color images. The original **ImageNet**

模型压缩：如第 3.4 节中所述，Clio 可以与最先进的压缩技术相结合。在我们的实现中，我们将 AutoML [17] 和 Meta Pruning [33] 应用于 MobileNetV2。我们还使用宽度乘数作为基线来压缩 MobileNetV2 模型。

减少训练时间：为了减少将 Clio 编译为 MobileNetV2 时的训练时间，我们为早期分割点选择每隔两个通道数，为后期分割点选择每隔四个通道数进行渐进切片。例如，当我们在实现中使用 300ms 的目标延迟时，MobileNetV2 第 3、5 或 8 层之后的分区点是根据我们的启发式选择的，因为它们可以将我们的带宽范围分别扩展到 195 kbps、175 kbps 和 98 kbps。

4.2 将模型编译到物联网设备

Clio 将模型编译为两个子模型——一个在物联网设备上本地执行的子模型，一个在云服务器上远程执行的子模型。我们看一下两个物联网处理器。GAP 8 是一款超低功耗神经加速器，具有可以加速卷积运算的硬件卷积引擎（HWCE）[51]。STM32 F7 是一款流行的基于 ARM7 的物联网处理器，没有卷积加速器 [6]。我们对 GAP8 加速器上的 MobileNetV2 的评估基于完整的端到端实现（下面将更详细地描述）。对于 ARM7 处理器，我们根据 GAP8 上的实现但使用 ARM7 频率和每条指令的 Risc-V 周期来估计计算成本。在 ARM7 中，乘法、加法和比较等运算在 MCU 中并行执行。

GAP8 实现：GAP8 平台仅对浅层模型和无线通信提供基本支持，我们使用较大的模型以及物联网和云模型的集成。我们实施了必要的驱动程序并对其进行了优化，以实现 BLE 的连续运行。我们还编写了软件来从 PyTorch/TFLite 获取模型并将其拆分/加载到 GAP8 上，GAP8 处理格式变化并对其进行优化以使其适合内存。

我们使用 TFLite [53] 文件在 GAP8 上进行模型部署。将模型转换为 TFLite 文件时，会融合一些操作（例如批量归一化）以进行部署。由于 GAP8 仅支持定点算术运算，因此在量化期间所有权重和输入都被转换为 8 位 Q 格式定点数。同时，中间结果也以 8 位定点数的形式存储和传输。

MobileNetV2 中有两种卷积，第一种是基本卷积，第二种是扩展卷积，扩展卷积又由三种基本卷积组成：扩展卷积、深度卷积和投影卷积。尽管 GAP8 具有硬件卷积引擎来加速卷积运算，但由于硬件设计的原因，某些运算并不支持，而是在计算核心上并行执行。此外，ReLU6 和 padding 等操作会在计算核心上融合并并行执行。

在硬件方面，我们将 GAP8 板与 HIMAX 摄像头、LCD 屏幕和通过 UART 接口连接的无线电屏蔽集成在一起（如图 5 所示）。集成板捕获

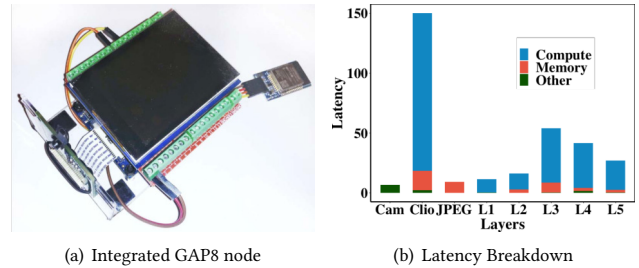


图 5: (a) 集成 GAP8 节点与灰度 HIMAX HM01B0 摄像头、B LE 扩展板和 LCD 显示屏, (b) 基于经验测量的延迟规格。

来自相机的图像，将其显示在液晶屏幕上，执行模型的早期层，然后通过无线电传输中间结果。

云执行：云中的执行速度比物联网设备快得多。对于我们的基准测试，我们使用 Xeon Silver 4116 2.10 GHz、32 GB DDR4 2400 MHz 和 GTX 1080 Ti GPU。

实时操作演示：我们集成了以上几部分，以支持连续实时操作。我们为系统制作了完整的视频演示（请参阅代码 [2]）——为了制作演示，我们必须针对来自 GAP8 板的 HIMAX 相机的灰度格式重新训练和优化模型，这与我们用于结果的 ImageNet/CIFAR-10 数据不同。

在 GAP8 上对 MobileNetV2 进行基准测试：我们对 GAP8 进行了分析，以获得不同计算块的延迟和能量基准。

对于延迟分析，我们使用硬件计数器来评估计算、内存复制和其他操作的总周期。我们通过除以 GAP8 时钟频率设置将总周期转换为延迟。图 5 显示了执行 MobileNetV2 早期层的 GAP8 实现所消耗的延迟细目。根据延迟细分，我们估计执行 MobileNetV2 时 GAP8 的等效计算能力约为 525 MMAC/s。

对于无线，我们发现从 GAP8 到 BLE 扩展板传输数据的最大速度为 250 kbps（由于 UART 速度限制）。在评估延迟和能量时，我们会在评估中使用这些数字。

5 评价

我们首先描述我们使用的数据集和我们比较的方案，然后再根据这些方法评估 Clio。

5.1 数据集

我们用于评估的两个主要数据集基于 CIFAR-10 和 ImageNet——这些数据集具有不同的图像大小，并展示了我们的方法在不同工作负载下的性能。CIFAR-10 [25] 数据集由 10 个类别的 60000 张 32×32 彩色图像组成，每个类别 6000 张图像。有 50000 张训练图像和 10000 张测试图像。ImageNet [14] 数据集有 224 个 $\times 224$ 色图像。原始的 ImageNet

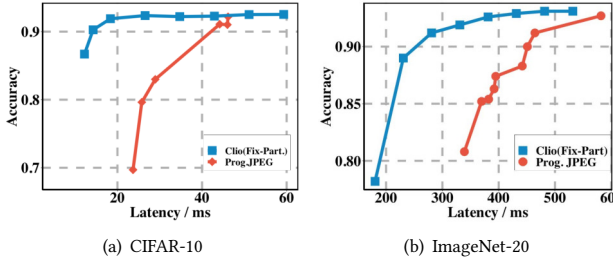


Figure 6: Prog. Slicing vs Prog. JPEG for CIFAR-10 and ImageNet-20 datasets. Latency corresponds to GAP8 and BLE radio (numbers in §4.2). Prog. Slicing has significantly better performance than Prog. JPEG for both CIFAR10 and ImageNet-20 datasets.

dataset consists of 1000 classes. In practice, however, IoT devices are typically focused on specific tasks. Rather than classify over 1000 classes, so we create a smaller dataset, ImageNet-20, with a random set of 20 classes and use this for our evaluation. We have 26000 images for training in 20 classes with 1300 images per class in training; 1000 images for evaluation.

5.2 Comparisons

We compare against several methods in this evaluation.

Data Compression: In terms of **data compression**, we compare **Clio** against transmission of JPEG-compressed raw data to the cloud for remote processing. We use a state-of-art JPEG-based encoder, DeepN-JPEG [32], which is specifically engineering for deep learning workloads and therefore performs better than traditional JPEG for deep learning tasks.

We evaluate against two versions of JPEG: a) *Baseline JPEG* where the image is compressed to a desired quality and the coefficients are sequentially transmitted, and b) *Progressive JPEG* where the compressed coefficients are interleaved in different passes of progressively higher detail.

Model Compression: In terms of **model compression**, we compare against several state-of-art local compression options: meta-pruning [33], adapting the width-multiplier and AutoML [17].

Partitioning: Finally, we also compare against state-of-art model partitioning methods including Neurosurgeon [21], JALAD [29], and 2-step Pruning [41].

5.3 Prog. Slicing versus Prog. JPEG

We now compare Progressive Slicing in **Clio** for a fixed choice of partition point (layer 5) against Progressive JPEG that uses the state-of-art DeepN-JPEG encoder [32]. Both operate in an interleaved manner and can stop at any time.

We first look at the performance benefits when compared with remote computation on JPEG-compressed data. Figure 6 shows latency results for two datasets (CIFAR-10 and ImageNet-20) and a BLE radio operating at 250kbps. We see that **Clio** with fixed partition point clearly outperforms Progressive JPEG, particularly at low latencies. Results are almost identical when the metric is energy instead of latency, hence we do not show these results.

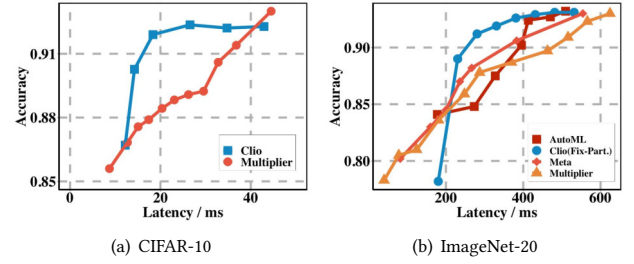


Figure 7: Clio vs Model Compression. Latency corresponds to GAP8 and BLE radio @ 250kbps. On CIFAR10, only width multipliers are effective and Clio is better than this method. On ImageNet-20, Clio is better than AutoML, meta-pruning and width multipliers except at low latencies where there is insufficient time for communication.

5.4 Clio versus Model Compression

We now evaluate the benefits of **Clio** over model compression approaches that seek to squeeze the entire MobileNetV2 model into the resource constraints of the GAP8.

Figure 7 shows the results. For small image sizes in CIFAR-10, **Clio** is significantly better than the width multiplier. The results for ImageNet-20 are more instructive. At low latency, we see that local compression methods generally perform better as expected since **Clio** can only transmit a very small amount of intermediate results to the cloud, which diminishes accuracy. Once latency increases above about 220ms, **Clio** performs better than all local model compression methods. Finally, when the latency is sufficiently high, the techniques start to converge since there is sufficient compute and sufficient data transfer for both local and remote processing to work well.

We note that these results are only based on progressive slicing and do not adapt the partition point. Augmenting progressive slicing with adaptive partitioning further improves the benefits of **Clio** over model compression (we defer this discussion to §5.6)

5.5 Comparing Local, Remote, and Clio

So far, we have separately evaluated **Clio** against model compression and data compression. We now look more holistically at these three methods to try to understand the operating regimes where each of them works best. The design space, however, is rather large since different factors affect the different methods — local execution depends on compute/memory resources and compute efficiency; remote execution depends on available bandwidth and radio efficiency; and **Clio** depends on all of the above factors. Rather than looking at each design parameter separately, we illustrate the general trends and tradeoffs by taking a few example architectures and the MobileNetV2 model.

We now compare end-to-end latency across a few exemplar architectures corresponding to different processors (GAP8 – 50mW @ 150MHz and STM32F7 ARM7 – 60mW @ 200MHz), radio power/bandwidth combinations (LoRa – 50kbps @ 60mW, ZigBee – 200kbps @ 60mW, BLE – 250kbps @ 15mW, WiFiax – 1 Mbps @ 100 mW), and image sizes (CIFAR-10 vs ImageNet-20). Since we cannot show the

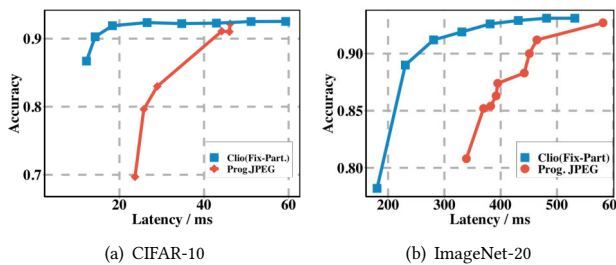


图 6：程序。切片与编程。CIFAR-10 和 ImageNet-20 数据集的 JPEG。延迟对应于 GAP8 和 BLE 无线电（第 4.2 节中的数字）。程序。切片的性能明显优于 Prog。CIFAR10 和 ImageNet-20 数据集的 JPEG。

数据集由 1000 个类组成。然而，在实践中，物联网设备通常专注于特定任务。我们没有对 1000 多个类进行分类，而是创建了一个较小的数据集 ImageNet-20，其中随机包含 20 个类，并将其用于我们的评估。我们有 26000 张图像用于 20 个类别的训练，每个类别有 1300 张图像进行训练；1000 张图像用于评估。

5.2 比较

我们在此评估中对几种方法进行了比较。

数据压缩：在数据压缩方面，我们将 Clío 与将 JPEG 压缩的原始数据传输到云端进行远程处理进行比较。我们使用最先进的基于 JPEG 的编码器 DeepN-JPEG [32]，它专门针对深度学习工作负载而设计，因此在深度学习任务中比传统的 JPEG 表现更好。

我们针对两个版本的 JPEG 进行评估：a) 基线 JPEG，其中图像被压缩到所需的质量，并且按顺序传输系数；b) 渐进式 JPEG，其中压缩系数在逐渐更高细节的不同通道中交错。

模型压缩：在模型压缩方面，我们与几种最先进的本地压缩选项进行比较：元剪枝[33]、调整宽度乘数和 AutoML [17]。

分区：最后，我们还与最先进的模型分区方法进行比较，包括 Neurosurgeon [21]、JALAD [29] 和 2-step Pruning [41]。

5.3 程序。切片与编程。JPEG

现在，我们将 Clío 中固定选择分区点（第 5 层）的渐进式切片与使用最先进的 DeepN-JPEG 编码器的渐进式 JPEG 进行比较 [32]。两者都是交错运行，并且可以随时停止。

我们首先看看与 JPEG 压缩数据的远程计算相比的性能优势。图 6 显示了两个数据集（CIFAR-10 和 ImageNet-20）和以 250kbps 运行的 BLE 无线电的延迟结果。我们看到具有固定分区点的 Clío 明显优于渐进式 JPEG，特别是在低延迟时。当指标是能量而不是延迟时，结果几乎相同，因此我们不显示这些结果。

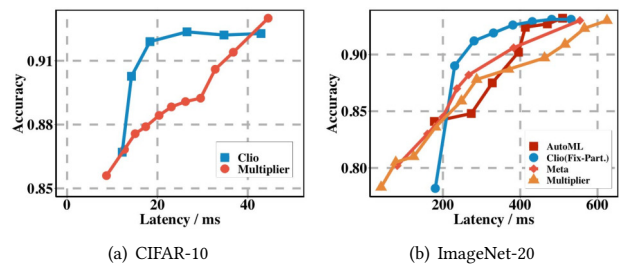


图 7：Clío 与模型压缩。延迟对应于 GAP8 和 BLE 无线电 @ 250kbps。在 CIFAR10 上，只有宽度乘数有效，Clío 比该方法更好。在 ImageNet-20 上，Clío 优于 AutoML、元剪枝和宽度乘数，除非在低延迟时没有足够的时间进行通信。

5.4 Clío 与模型压缩

我们现在评估 Clío 相对于模型压缩方法的优势，模型压缩方法试图将整个 MobileNetV2 模型压缩到 GAP8 的资源限制中。

图 7 显示了结果。对于 CIFAR-10 中的小图像尺寸，Clío 明显优于宽度乘数。ImageNet-20 的结果更具指导意义。在低延迟时，我们发现本地压缩方法通常表现得更好，因为 Clío 只能将极少量的中间结果传输到云，这会降低准确性。一旦延迟增加到约 220 毫秒以上，Clío 的性能就优于所有本地模型压缩方法。最后，当延迟足够高时，这些技术开始收敛，因为有足够的计算和足够的数据传输来保证本地和远程处理正常工作。

我们注意到这些结果仅基于渐进切片并且不适应分割点。通过自适应分区增强渐进式切片进一步提高了 Clío 相对于模型压缩的优势（我们将讨论推迟到第 5.6 节）

5.5 比较本地、远程和 Clío

到目前为止，我们已经分别针对模型压缩和数据压缩评估了 Clío。我们现在更全面地研究这三种方法，试图了解每种方法最有效的操作机制。然而，设计空间相当大，因为不同的因素影响不同的方法——本地执行取决于计算/内存资源和计算效率；远程执行取决于可用带宽和无线电效率；和 Clío 取决于上述所有因素。我们没有单独查看每个设计参数，而是通过一些示例架构和 MobileNetV2 模型来说明一般趋势和权衡。

现在，我们比较了与不同处理器（GAP8 – 50mW @ 150MHz 和 S TM32F7 ARM7 – 60mW @ 200MHz）、无线电功率/带宽组合（LoRa – 50kbps @ 60mW、ZigBee – 200kbps @ 60mW、BLE – 250kbps @ 15mW、WiFiax – 1 Mbps @ 100 mW）和图像大小（CIFAR-10 与 ImageNet-20）。由于我们无法展示

entire envelope for each setting, we look at the performance ratio between each method and **Clio** corresponding to 90% accuracy (trends are similar for other choices of accuracy). For each method, we evaluate the ratio of its performance against **Clio** — so a latency or energy ratio greater than one means that the method has higher latency/energy compared to **Clio**.

Figure 8 shows the results for CIFAR-10 and ImageNet-20 — the caption describes how to interpret the table and color code. We see that as we go from left to right (low to high bandwidth), performance gains of **Clio** over remote processing diminishes whereas performance gains over local processing improves. As we go from a general-purpose ARM7 processor to a specialized GAP8 accelerator, local processing improves in performance. We see a significant band of configurations where **Clio** has better performance than local and remote processing. The results are intuitive and generally follows the trend of available compute and bandwidth resources — when compute is more plentiful than bandwidth, local compute is better; when bandwidth is more than local compute resources, remote computation is better and **Clio** when there is roughly equivalent compute and bandwidth resources hence using both together provides advantages.

5.6 Benefits of adapting the partition point

So far, our results have focused exclusively on progressive slicing. We now turn to optimizing the selection of the partition point together with progressive slicing. We consider two versions of **Clio** — a fixed partition version as before (layer 5) and an adaptive version that uses the average bandwidth over the last minute to determine what partition point to use (as described in §3.3). We also show the performance of two versions of JPEG — Progressive JPEG which makes no bandwidth assumptions and Baseline JPEG which compresses based on the recent bandwidth history.

We look at performance across a range of dynamic bandwidth scenarios on synthetic and real traces to allow us to understand the performance of **Clio** in a dynamic environment. We focus on ImageNet-20 images due to their larger size and consider the case where images need to be processed within 300ms.

Synthetic Trace-driven Evaluation: To demonstrate the ability of **Clio** to adapt to dynamics, we use synthetic bandwidth traces where we systematically vary the amount of dynamics. The mechanism for constructing traces is based on the procedure in [34] and uses a Markovian model in which each state represented an average bandwidth in the selected range. State transitions were performed at a 1 second granularity and followed a geometric distribution. Each bandwidth value was then drawn from a Gaussian distribution centered around the average bandwidth for the current state, with variance uniformly distributed in $[0.05, \sigma_{MAX}]$. We vary σ_{MAX} to control the amount of variance in the trace and plot the performance of different schemes to show how they adapt to dynamics. Our synthetic dataset covers typical wireless network conditions for low-power radios, with average bandwidth ranging from 50 Kbps to 1 Mbps.

Figure 9 shows how gracefully different methods degrade in accuracy as a result of divergence between the real bandwidth and estimated bandwidth. The larger this gap is, the more likely it is that the partition point or compression ratio is improperly chosen.

We see that **Clio** with adaptive partitioning has the least degradation in accuracy as the bandwidth variability increases. The accuracy degradation for Baseline JPEG is 2–3 $\times$ worse than that for **Clio** with adaptive partitioning. In fact, even **Clio** with a fixed partition point performs marginally better than Baseline JPEG albeit. Progressive JPEG performs significantly worse than the **Clio**-based methods.

Real-world Trace-driven Evaluation: We now evaluate **Clio** across several real-world bandwidth traces. Specifically, we look at HSDPA mobility traces which were collected in different mobile environments such as buses, trains, and ferries [37].

Figure 10 shows the results. We note that when the bandwidth is less than the minimum bandwidth required for the technique used (e.g. first scan/slice) to generate a prediction result, then we record the corresponding accuracy as zero. We see that Progressive Slicing is better than Progressive JPEG across all traces; in fact, our method is also comparable to Baseline JPEG even though the method makes no assumptions about available bandwidth. **Clio** augmented with partition point adaptation improves performance and is as good or better than JPEG-based methods.

We expand on the above results in two ways. First, we take all the instances from the trace where there the predicted bandwidth does not match the actual bandwidth, and look at what effect that has on accuracy. Figure 11(a) shows the loss in accuracy for different methods across different gaps between predicted and actual bandwidths. We see that **Clio** with adaptive partitioning can achieve the best overall accuracy and minimal accuracy degradation even when there is a substantial gap between real and estimation bandwidths result from network dynamics.

Second, we illustrate the difference between adaptive partitioning versus fixed partitioning using a time-series snippet from the traces with bandwidth dynamics. We see that adaptive partitioning adapts to larger bandwidth changes effectively to minimize the loss in performance due to dynamics.

Performance under known bandwidth: In the trace-driven evaluation, we had to predict bandwidth based on past measurements; now, we take a more favorable scenario for Baseline JPEG where bandwidth is known and quality of compressed JPEG is optimized for the given bandwidth. These results are for the ImageNet-20 dataset.

Figure 12 shows the results. We see that choosing the optimal partition point improves the classification performance particularly at lower latencies. We see that Baseline JPEG compression is significantly better than Progressive JPEG. The graph also shows interesting tradeoffs between Baseline JPEG versus **Clio**. We see that for very low latency, Baseline JPEG is superior since JPEG computation has lower latency than that incurred by **Clio** to compute the initial layers. As available latency increases above a minimum threshold to execute a few layers locally, **Clio** starts to perform better. Both the progressive-only and optimal-partitioning + progressive-based versions of **Clio** perform better than JPEG-based options. Once the latency is long enough to transmit sufficient data, all approaches converge in performance.

Compute overhead of switching partition: Changing the partition point incurs some additional overhead since the IoT-portion of the model needs to be changed. We assume that the different

对于每个设置的整个包络，我们查看每种方法与 Clio 之间对应于 90% 准确度的性能比（其他准确度选择的趋势类似）。对于每种方法，我们都会评估其与 Clio 的性能比率 - 因此，延迟或能量比率大于 1 意味着该方法与 Clio 相比具有更高的延迟/能量。

图 8 显示了 CIFAR-10 和 ImageNet-20 的结果 - 标题描述了如何解释表格和颜色代码。我们看到，当我们从左到右（从低到高带宽）时，Clio 相对于远程处理的性能增益会减少，而相对于本地处理的性能增益会有所提高。当我们从通用 ARM7 处理器转向专用 GAP8 加速器时，本地处理的性能得到了提高。我们看到很多配置中 Clio 的性能优于本地和远程处理。结果很直观，并且通常遵循可用计算和带宽资源的趋势——当计算比带宽更充足时，本地计算更好；当计算比带宽更充足时，本地计算更好；当带宽大于本地计算资源时，远程计算会更好，而当计算和带宽资源大致相当时，Clio 会更好，因此同时使用两者会带来优势。

5.6 调整分区点的好处

到目前为止，我们的结果仅集中于渐进切片。我们现在转向优化分割点的选择以及渐进切片。我们考虑 Clio 的两个版本 - 一个是之前的固定分区版本（第 5 层），另一个是自适应版本，该版本使用最后一分钟的平均带宽来确定要使用的分区点（如第 3.3 节中所述）。我们还展示了 JPEG 的两个版本的性能 - 不做带宽假设的渐进式 JPEG 和基于最近带宽历史记录进行压缩的基线 JPEG。

我们在合成和真实轨迹上查看一系列动态带宽场景的性能，以便我们了解 Clio 在动态环境中的性能。我们关注 ImageNet-20 图像，因为它们尺寸较大，并考虑图像需要在 300ms 内处理的情况。

合成轨迹驱动的评估：为了证明 Clio 适应动态的能力，我们使用合成带宽轨迹，系统地改变动态量。构建轨迹的机制基于 [34] 中的过程，并使用马尔可夫模型，其中每个状态代表所选范围内的平均带宽。状态转换以 1 秒的粒度执行，并遵循几何分布。然后从以当前状态的平均带宽为中心的高斯分布中得出每个带宽值，方差均匀分布在 $[0.05, \sigma_{MAX}]$ 中。我们改变 σ_{MAX} 来控制迹线中的方差量，并绘制不同方案的性能以显示它们如何适应动态。我们的合成数据集涵盖了低功耗无线的典型无线网络条件，平均带宽范围从 50 Kbps 到 1 Mbps。

图 9 显示了由于实际带宽和估计带宽之间的差异，不同的方法如何优雅地降低精度。这个差距越大，分区点或压缩比选择不当的可能性就越大。

我们发现，随着带宽变化的增加，具有自适应分区的 Clio 的准确性下降最小。Baseline JPEG 的精度下降比具有自适应分区的 Clio 差 2-3x。事实上，即使具有固定分割点的 Clio 的性能也比 Baseline JPEG 稍好一些。渐进式 JPEG 的性能明显比基于 Clio 的方法差。

真实世界跟踪驱动的评估：我们现在通过多个真实世界带宽跟踪来评估 Clio。具体来说，我们研究了在公共汽车、火车和渡轮等不同移动环境中收集的 HSDPA 移动轨迹 [37]。

图 10 显示了结果。我们注意到，当带宽小于生成预测结果所使用技术（例如第一次扫描/切片）所需的最小带宽时，我们将相应的精度记录为零。我们发现渐进式切片在所有轨迹上都优于渐进式 JPEG；事实上，我们的方法也与 Baseline JPEG 相当，尽管该方法没有对可用带宽做出任何假设。通过分区点自适应增强的 Clio 提高了性能，并且与基于 JPEG 的方法一样好甚至更好。

我们通过两种方式扩展上述结果。首先，我们从跟踪中获取预测带宽与实际带宽不匹配的所有实例，并查看这对准确性有何影响。图 11 (a) 显示了不同方法在预测带宽和实际带宽之间的不同差距上的准确性损失。我们看到，即使网络动态导致的实际带宽和估计带宽之间存在很大差距，具有自适应分区的 Clio 也可以实现最佳的整体精度和最小的精度下降。

其次，我们使用来自带宽动态轨迹的时间序列片段来说明自适应分区与固定分区之间的区别。我们看到自适应分区可以有效地适应更大的带宽变化，从而最大限度地减少动态造成的性能损失。

已知带宽下的性能：在跟踪驱动的评估中，我们必须根据过去的测量来预测带宽；现在，我们对基线 JPEG 采取更有利的方案，其中带宽已知，并且针对给定带宽优化了压缩 JPEG 的质量。这些结果适用于 ImageNet-20 数据集。

图 12 显示了结果。我们发现，选择最佳划分点可以提高分类性能，尤其是在较低延迟的情况下。我们发现基线 JPEG 压缩明显优于渐进式 JPEG。该图还显示了 Baseline JPEG 与 Clio 之间有趣的权衡。我们看到，对于非常低的延迟，Baseline JPEG 更优越，因为 JPEG 计算的延迟低于 Clio 计算初始层所产生的延迟。随着可用延迟增加到本地执行几层的最小阈值以上，Clio 开始表现得更好。仅渐进式版本和基于最佳分区 + 渐进式版本的 Clio 的性能均优于基于 JPEG 的选项。一旦延迟足够长以传输足够的数据，所有方法都会在性能上收敛。

切换分区的计算开销：由于需要更改模型的物联网部分，因此更改分区点会产生一些额外的开销。我们假设不同的

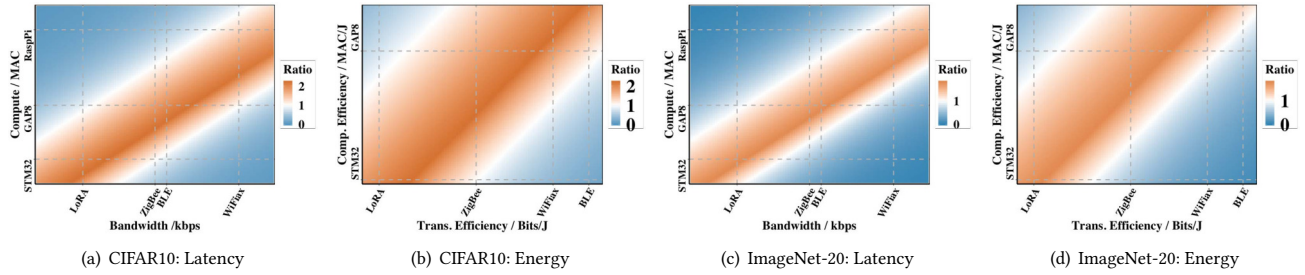


Figure 8: Heatmaps representing the regimes where local, remote and partitioned computation offer benefits. The left two graphs correspond to CIFAR-10 and the right two correspond to ImageNet-20. The band in the middle of each graph corresponds to the regime where Clío is the best approach; to the left of the band is the region where remote computation (on jpeg-compressed data) is better and the right of the band is the region where local computation (via a compressed model) is more efficient.

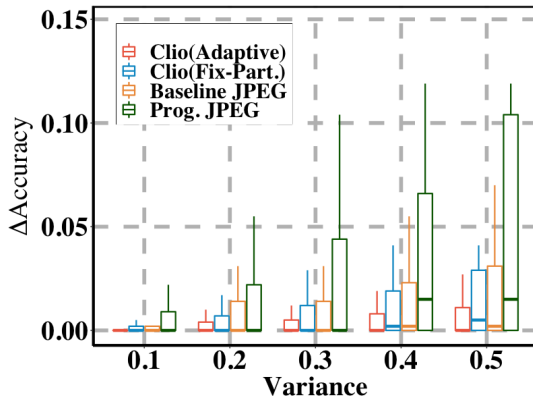


Figure 9: We use synthetic traces and gradually increase variance in bandwidth to show robustness of different methods. Clío with adaptive partitioning is most robust to bandwidth variations and degrades more gracefully than Baseline JPEG.

models are stored on the GAP8 flash and the cost only involves switching between the models. On the GAP8, the weights of the model are stored as files in HyperFlash and need to be copied to cache or HyperRAM before the model can be run. Therefore, when a partition point change is triggered, we need to first free the cache and HyperRAM for the current model and allocate new space for the new model. We empirically evaluated that this process only takes around 20ms on the GAP8, which is a small overhead given relatively infrequent switching between partition points (for example, the 20ms overhead is barely visible in Figure 11(b)).

5.7 Evaluation over GAP8 Hardware

We now look at an end-to-end evaluation over the actual GAP8 platform and BLE radio. One key difference between the results so far and our real experiments over the GAP8 is the effect of radio duty-cycling. So far, we have assumed that the radio can be instantly connected to the cloud whenever needed. This is, of course, unrealistic for duty-cycled radios — for example, a BLE sender and receiver wakeup at periodic intervals to see if there is

new data to transmit and this interval determines how quickly data transmission can begin. The choice of wakeup interval influences duty-cycling efficiency — if the interval is high, then the radio consumes less power (e.g. BLE consumes only 45μW if connection latency is 1 second), and if the interval is low, frequent wakeups increase power consumption (e.g. BLE consumes 460μW at 40ms connection latency) [46].

Figure 13 shows the results when we use BLE 4.0 and BLE 4.2 as the radio. The difference between these is that BLE 4.0 can only transmit data via notification packets whose maximum size is 20 bytes while BLE 4.2 enlarges the maximum size to 251 bytes. We see that the end-to-end latency of Clío increases very little even as connection interval increases significantly from 7.5ms to 40ms whereas the end-to-end latency for JPEG increases steeply in a step-wise manner (corresponding to each additional layer being transmitted). One advantage for Clío in the presence of connection latency is that since Clío processes several initial layers before transmitting to the cloud, it is able to mask a significant fraction of the end-to-end latency.

Thus, an important advantage of Clío in the context of deployments that use duty-cycled radios is that Clío is effective at masking the wake-up delays by locally processing the data until the connection becomes available. In contrast, the computation time of JPEG is small and does not benefit as much from the wake-up delay.

5.8 Other models and methods

We now look at how Clío performs for models other than MobileNetV2 and compare against several other partitioning methods.

Compatibility with other models and techniques: So far, we have looked only at MobileNetV2 but Clío can also be used with other deep learning models. Figure 14 compares Progressive Slicing, Progressive JPEG and Model Compression for another popular model, ResNet [16]. Since ResNet has higher compute requirements, the bandwidth point at which it becomes better than remote computation is lower than for MobileNetV2. We generally see that ResNet is most effective at bandwidth lower than 200kps, so we show results for ResNet when the wireless interface is a LoRa radio operating at 50kpbs. For this radio, we see that the latency envelope from ResNet is better than local and remote computation. We see

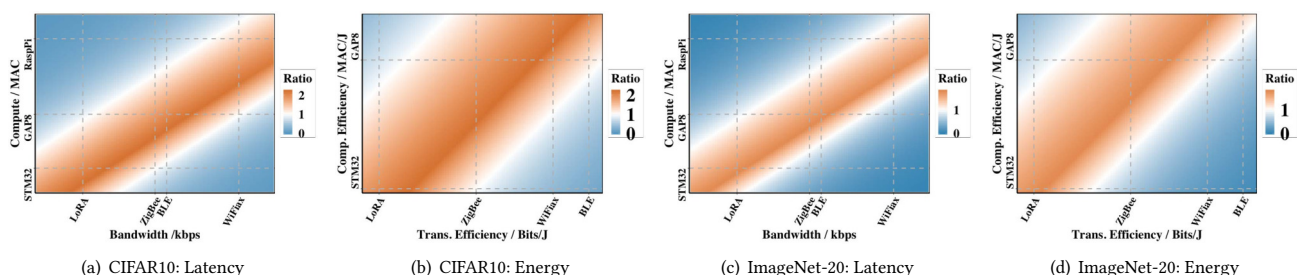


图 8：热图代表本地、远程和分区计算提供优势的情况。左边两张图对应于 CIFAR-10，右边两张图对应于 ImageNet-20。每张图中间的带对应于 Clío 是最佳方法的状态；带的左侧是远程计算（在 jpeg 压缩数据上）更好的区域，带的右侧是本地计算（通过压缩模型）更有效的区域。

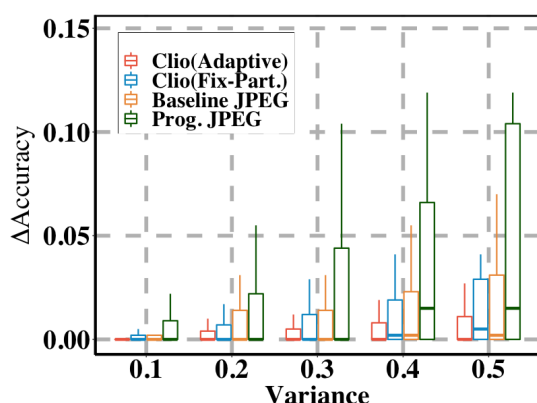


图 9：我们使用合成迹线并逐渐增加带宽方差，以显示不同方法的稳健性。具有自适应分区的 Clío 对带宽变化的鲁棒性最强，并且比 Base-line JPEG 的降级更平稳。

模型存储在 GAP8 闪存上，成本仅涉及模型之间的切换。在 GAP8 上，模型的权重以文件形式存储在 HyperFlash 中，需要将其复制到缓存或 HyperRAM 中，然后才能运行模型。因此，当触发分区点更改时，我们需要首先释放当前模型的缓存和 HyperRAM，并为新模型分配新的空间。我们根据经验评估，这个过程在 GAP8 上只需要大约 20ms，考虑到分区点之间的切换相对不频繁，这是一个很小的开销（例如，20ms 的开销在图 11 (b) 中几乎不可见）。

5.7 GAP8 硬件评估

我们现在看看对实际 GAP8 平台和 BLE 无线电的端到端评估。迄今为止的结果与我们在 GAP8 上进行的实际实验之间的一个关键区别是无线电占空比的影响。到目前为止，我们假设无线电可以在需要时立即连接到云端。当然，这对于占空比无线电来说是不现实的——例如，BLE 发送器和接收器定期唤醒以查看是否有

要传输的新数据，此间隔决定了数据传输开始的速度。唤醒间隔的选择会影响占空比效率——如果间隔较高，则无线电消耗的功率较少（例如，如果连接延迟为 1 秒，BLE 仅消耗 $45\mu\text{W}$ ），如果间隔较低，则频繁唤醒会增加功耗（例如，在 40 ms 连接延迟时，BLE 消耗 $460\mu\text{W}$ ）[46]。

图 13 显示了我们使用 BLE 4.0 和 BLE 4.2 作为无线电时的结果。两者的区别在于，BLE 4.0 只能通过最大大小为 20 字节的通知数据包传输数据，而 BLE 4.2 将最大大小扩大到 251 字节。我们看到，即使连接间隔从 7.5 毫秒显著增加到 40 毫秒，Clío 的端到端延迟也几乎没有增加，而 JPEG 的端到端延迟以逐步方式急剧增加（对应于传输的每个附加层）。在存在连接延迟的情况下，Clío 的一个优势是，由于 Clío 在传输到云之前处理多个初始层，因此它能够掩盖很大一部分端到端延迟。

因此，在使用占空比无线电的部署环境中，Clío 的一个重要优势是，Clío 通过在本地处理数据直至连接可用，可以有效地屏蔽唤醒延迟。相比之下，JPEG 的计算时间较小，并且无法从唤醒延迟中获益。

5.8 其他模型和方法

我们现在看看 Clío 在 MobileNetV2 以外的模型上的表现如何，并与其他几种划分方法进行比较。

与其他模型和技术的兼容性：到目前为止，我们只研究了 MobileNetV2，但 Clío 也可以与其他深度学习模型一起使用。图 14 比较了另一种流行模型 ResNet [16] 的渐进式切片、渐进式 JPEG 和模型压缩。由于 ResNet 具有更高的计算要求，因此它优于远程计算的带宽点低于 MobileNetV2。我们通常认为 ResNet 在带宽低于 200kps 时最为有效，因此我们展示了当无线接口是以 50kbps 运行的 LoRa 无线电时 ResNet 的结果。对于该无线电，我们看到 ResNet 的延迟包络优于本地和远程计算。我们看到

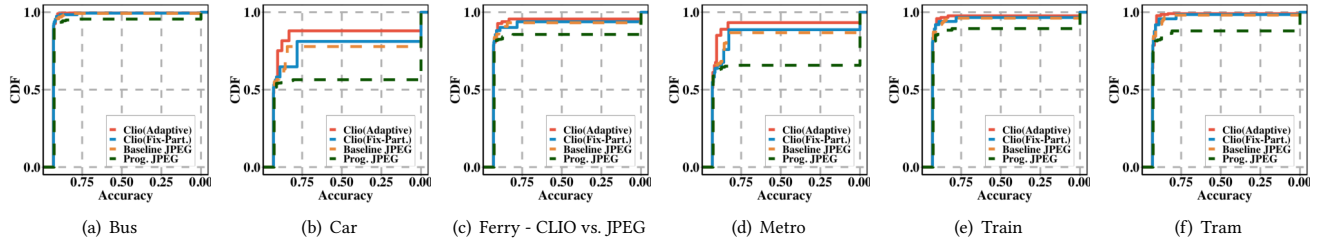


Figure 10: CDF of performance of Clío and JPEG for different mobility traces.

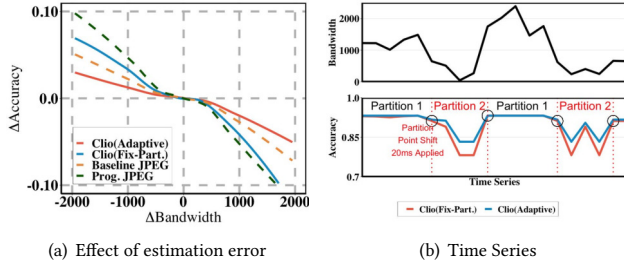


Figure 11: (a) Even when there is a large gap between estimated and actual bandwidth, Clío with partition point adaptation performs best. (b) Time series showing bandwidth changes and partition point adaptation to minimize loss in accuracy

similar trends with VGG-19 which is also a large model but do not show results due to space constraints.

Another technique to reduce communication is to use early exit which allows the model to stop execution at the IoT device if the prediction result can be predicted with high confidence [44, 45]. This technique is compatible with **Clío** and early exit can be folded into **Clío** pipeline. However, we found that early exit at the IoT device has poor accuracy (less than 70%) for models like MobileNetV2 given the IoT device constraints so we did not incorporate this into **Clío**.

Comparison against JALAD: Figure 15 shows results of our comparison against JALAD[29], a technique for reducing the size of intermediate results by quantizing with fewer bits. **Clío** performs better across the entire range, particularly at low compression ratios. This is because **Clío** can leverage the sparsity of the intermediate results while JALAD only leverages the sparsity in the fixed-point representation. We note that the quantization optimizations in JALAD can also be incorporated with **Clío** to improve results. For example, performance is unaffected by 6-bit quantization as opposed to 8-bit quantization and this can help improve **Clío**'s results.

Comparison against NeuroSurgeon: Table 1 compares **Clío** against Neurosurgeon across different radios and presents two numbers — the first is for similar accuracy as Neurosurgeon (i.e. less than 0.5% accuracy loss) and the second is for lower accuracy (roughly 3% accuracy loss). We see that even at almost identical

	Bandwidth			
	LoRA	ZigBee	BLE	WiFi
STM32F7	2.3x 3.6x	1.5x 1.8x	1.3x 1.7x	1.1x 1.2x
GAP 8	2.8x 6.3x	2.2x 3.4x	2.0x 3.0x	1.3x 1.5x

Table 1: Comparison against Neurosurgeon on end-to-end latency on ImageNet-20.

	Bandwidth			
	LoRA	ZigBee	BLE	WiFi
STM32F7	1.65x	1.39x	1.32x	1.08x
GAP 8	1.64x	1.64x	1.63x	1.27x

Table 2: Comparison against 2-step pruning on end-to-end latency on VGG/CIFAR-10.

accuracy, **Clío** performs better since it uses progressive slicing and adaptive partitioning to improve performance. **Clío** provides significantly better performance if the application can tolerate additional accuracy loss.

Comparison against 2-step Pruning: We also compare **Clío** with a two-step pruning method proposed in [41]. This method prunes the original model in two passes and partitions the pruned model into two parts (edge and cloud). In the first pruning pass, all layers of the model are pruned by using state-of-art pruning methods; in the second pass, each layer is pruned one-by-one while keeping other layers intact. Finally, the partition point is determined such that end-to-end latency requirements are met.

The 2-step pruning method is evaluated using VGGNet and CIFAR-10, so we compare with **Clío** for the same model/dataset. We compare end-to-end latency of these methods for comparable accuracy. Table 2 shows that **Clío** performs better than 2-step pruning across the different bandwidth settings.

Overall, we see that **Clío** performs better than alternate partitioning methods since the progressive slicing method manipulates the intermediate feature maps, which is critical for dealing with highly constrained network conditions.

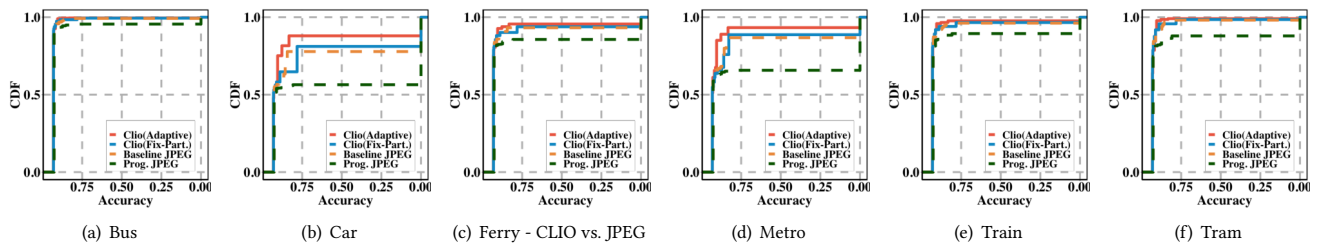


图 10: 不同手机上 Clío 和 JPEG 性能的 CDF

ty 痕迹。

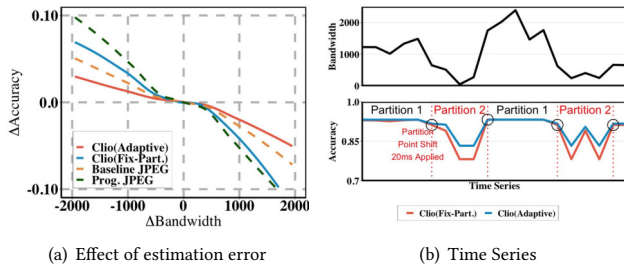


图 11: (a) 即使估计带宽与实际带宽之间存在很大差距，采用分区点自适应的 Clío 也能表现最佳。(b) 显示带宽变化和分区点适应的时间序列，以尽量减少准确性损失

与 VGG-19 类似的趋势，它也是一个大型模型，但由于空间限制没有显示结果。

另一种减少通信的技术是使用提前退出，如果可以以高置信度预测结果，则允许模型停止在 IoT 设备上执行 [44, 45]。该技术与 Clío 兼容，早期退出可以合并到 Clío 管道中。然而，我们发现，考虑到物联网设备的限制，对于像 MobileNet V2 这样的模型来说，物联网设备的早期退出准确率很差（低于 70%），因此我们没有将其合并到 Clío 中。

与 JALAD 的比较：图 15 显示了我们与 JALAD[29] 的比较结果，JALAD[29] 是一种通过使用更少的位数进行量化来减小中间结果大小的技术。Clío 在整个范围内表现更好，特别是在低压缩比下。这是因为 Clío 可以利用中间结果的稀疏性，而 JALAD 只能利用定点表示中的稀疏性。我们注意到 JALAD 中的量化优化也可以与 Clío 结合以改善结果。例如，与 8 位量化相比，性能不受 6 位量化的影响，这有助于改善 Clío 的结果。

与 NeuroSurgeon 的比较：表 1 对不同无线电中的 Clío 与 Neurosurgeon 进行了比较，并显示了两个数字 - 第一个是与 Neurosurgeon 相似的准确度（即小于 0.5% 的准确度损失），第二个是较低的准确度（大约 3% 的准确度损失）。我们看到，即使在几乎相同的情况下

	Bandwidth			
	LoRA	ZigBee	BLE	WiFi
STM32F7	2.3x 3.6x	1.5x 1.8x	1.3x 1.7x	1.1x 1.2x
GAP 8	2.8x 6.3x	2.2x 3.4x	2.0x 3.0x	1.3x 1.5x

表 1: 与神经外科医生在 ImageNet-20 上的端到端延迟的比较

	Bandwidth			
	LoRA	ZigBee	BLE	WiFi
STM32F7	1.65x	1.39x	1.32x	1.08x
GAP 8	1.64x	1.64x	1.63x	1.27x

表 2: VGG/CIFAR-10 上端到端延迟与两步修剪的比较。

准确性方面，Clío 表现更好，因为它使用渐进切片和自适应分区来提高性能。如果应用程序可以容忍额外的精度损失，Clío 可以提供明显更好的性能。

与两步剪枝的比较：我们还将 Clío 与[41]中提出的两步剪枝方法进行比较。该方法分两次修剪原始模型，并将修剪后的模型划分为两部分（边缘和云）。在第一遍剪枝中，模型的所有层都使用最先进的剪枝方法进行剪枝；在第二遍中，每一层都被一层一层地修剪，同时保持其他层完好无损。最后，确定划分点以满足端到端延迟要求。

使用 VGGNet 和 CIFAR-10 评估两步剪枝方法，因此我们与相同模型/数据集的 Clío 进行比较。我们比较这些方法的端到端延迟以获得可比较的精度。表 2 显示，Clío 在不同带宽设置下的性能优于两步修剪。

总的来说，我们看到 Clío 的性能优于其他分区方法，因为渐进切片方法操纵中间特征图，这对于处理高度受限的网络条件至关重要。

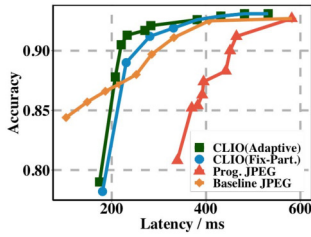


Figure 12: Adaptive Partitioning Benefits

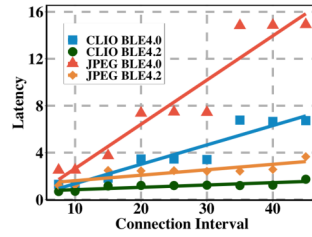


Figure 13: Experimental Results on GAP8

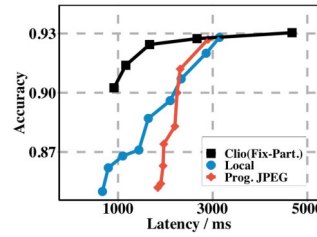


Figure 14: Clcio vs JPEG for ResNet18

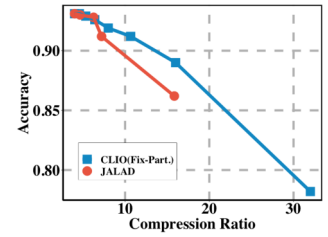


Figure 15: Comparison between Clcio and JALAD

6 RELATED WORK

Related efforts can be largely classified into three categories – deep learning for resource-constrained devices, cloud offload and partitioned execution.

Model compilation for single device: There has been much recent interest on optimizing deep learning models for embedded MCUs [8, 27, 28, 31, 56]. While most of this work assumes less stringent constraints than required by IoT devices, some specifically target such devices [27, 31, 56]. Model compression is a more mature field for smartphone-class devices – companies like Google and Amazon offer model compression tools to make it easy for developers to optimize deep learning models for mobile devices [47, 48]. Our work is complementary to these efforts and enables model compilation techniques to also leverage low-power wireless radios and compile large models to distributed IoT-Cloud networks.

Cloud offload: There has been more than a decade of work on optimizing cloud offload and this is one of the most common ways of deploying IoT systems [7, 11, 13, 19, 23, 30, 39]. Generally, cloud offload methods are agnostic to the specific computation performed at the edge (IoT) device versus the edge cloud. They primarily deal with variations in network delay and bandwidth at the network layer. In contrast, we develop an end-to-end method for compiling machine learning models such that they can seamlessly adapt to fluctuations in network conditions.

Partitioning analytics: Several recent efforts have looked at partitioning deep learning models across the cloud and edge (e.g. Neurosurgeon [21]). There has been some work that also extends the idea of partitioning – [29] quantizes and compresses intermediate results, [24] also lossily compresses intermediate results, and [40] describes additional model pruning of the local IoT model to reduce bandwidth. We show that progressive slices can provide benefits over compression of intermediate results and over model compression techniques.

7 CONCLUSIONS

In conclusion, we present a new approach to model compilation for low-power IoT where we automatically split a deep learning model between the IoT device and the cloud to optimize latency and energy while also being resilient to bandwidth dynamics. Our work is a departure from prior work in that **Clcio** operates in a progressive manner and simultaneously reasons about model compression (for local computation) and data compression (for cloud

computation). Our approach is end-to-end and optimizes overall inference performance while taking into consideration resource limitations such as bandwidth, energy, and computation ability. We believe that our approach is generalizable and paves the way for leveraging both low-power accelerators and low-power radios. It can be used in a host of IoT and sensor network deployments.

Our work also opens up several research directions. While we focused on images, we suspect similar methods will work for complex audio processing models, for example, methods to enable speech processing in noisy environments [4]. Also, we are currently working on extending **Clcio** to models for video analytics – these models are quite different since they learn temporal dependencies via LSTMs, 3D convolutions, or some combination of CNNs and optical flow tracking [20, 22, 43, 54, 57]. Our expectation is that as the complexity of these models increases, partitioning between edge and cloud becomes more essential and model compilation methods such as **Clcio** will be even more important.

ACKNOWLEDGMENTS

The research reported in this paper was sponsored in part by the CDC Army Research Laboratory (ARL) under Cooperative Agreement W911NF-17-2-0196 (ARL IoBT CRA) and by National Science Foundation under Grant No. 1719386 and No. 1815347. The views and conclusions contained in this document are those of the authors and should not be interpreted as representing the official policies, either expressed or implied, of the ARL, NSF or the U.S. Government. The U.S. Government is authorized to reproduce and distribute reprints for Government purposes notwithstanding any copyright notation herein.

REFERENCES

- [1] cc2531 zigbee chip. <http://www.ti.com/lit/ds/symlink/cc2531.pdf>.
- [2] Github repo. <https://github.com/jinhuang01/CLIO>.
- [3] Long range, low power rf transceiver 860-1000 mhz with lora technology. <https://www.semtech.com/products/wireless-rf/lora-transceivers/sx1272>.
- [4] Nsf: Hearables challenge. <https://www.nasa.gov/feature/nsf-hearables-challenge>.
- [5] St spbt2632c2a bluetooth module. <http://www.st.com/web/en/resource/technical/document/datasheet/DM00048919.pdf>.
- [6] Stm32f7: A very high-performance mcus with arm cortex-m7 core. <https://www.st.com/en/microcontrollers-microprocessors/stm32f7-series.html>.
- [7] A. Ashok, P. Steenkiste, and F. Bai. Enabling vehicular applications using cloud services through adaptive computation offloading. In *Proceedings of the 6th International Workshop on Mobile Cloud Computing and Services*, pages 1–7. ACM, 2015.
- [8] S. Bhattacharya and N. D. Lane. Sparsification and separation of deep learning layers for constrained resource inference on wearables. In *Proceedings of the 14th ACM Conference on Embedded Network Sensor Systems CD-ROM*, pages 176–189. ACM, 2016.
- [9] Bosch. BMI160: Ultra Low Power Inertial Measurement Unit.

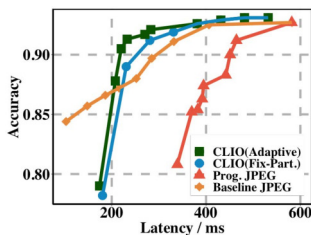


图 12: 自适应分区的优点

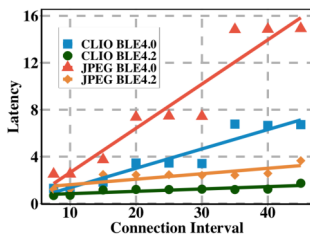


图 13: GAP8 的实验结果

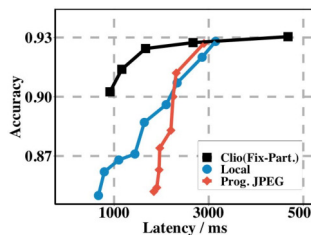


图 14: ResNet18 的 Clio 与 JPEG

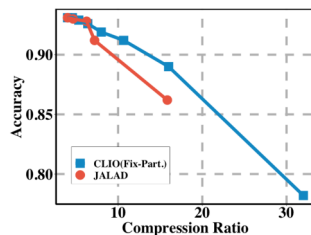


图 15: Clio 和 JALAD 之间的比较

6 相关工作

相关工作可大致分为三类——针对资源受限设备的深度学习、云卸载和分区执行。

单个设备的模型编译：最近人们对优化嵌入式 MCU 的深度学习模型产生了很大的兴趣 [8,27,28,31,56]。虽然大多数工作假设的约束不如物联网设备要求的严格，但有些工作专门针对此类设备 [27,31,56]。对于智能手机级设备来说，模型压缩是一个更加成熟的领域——谷歌和亚马逊等公司提供模型压缩工具，使开发人员可以轻松优化移动设备的深度学习模型 [47, 48]。我们的工作是对这些努力的补充，使模型编译技术也能够利用低功耗无线电并将大型模型编译到分布式物联网云网络。

云卸载：优化云卸载的工作已经有十多年了，这是部署物联网系统的最常见方法之一 [7,11,13,19,23,30,39]。一般来说，云卸载方法与边缘 (IoT) 设备与边缘云执行的特定计算无关。它们主要处理网络层的网络延迟和带宽的变化。相比之下，我们开发了一种用于编译机器学习模型的端到端方法，以便它们可以无缝适应网络条件的波动。

分区分析：最近的几项工作着眼于跨云和边缘对深度学习模型进行分区 (例如 Neurosurgeon [21])。有一些工作也扩展了分区的思想——[29]量化和压缩中间结果，[24]也有损地压缩中间结果，[40]描述了本地物联网模型的额外模型修剪以减少带宽。我们证明渐进切片可以提供优于中间结果压缩和模型压缩技术的优势。

7 结论

总之，我们提出了一种低功耗物联网模型编译的新方法，在该方法中，我们自动在物联网设备和云之间拆分深度学习模型，以优化延迟和能源，同时还能适应带宽动态。我们的工作与之前的工作不同，Clio 以渐进的方式运行，同时解释模型压缩（用于本地计算）和数据压缩（用于云）

计算)。我们的方法是端到端的，优化整体推理性能，同时考虑带宽、能量和计算能力等资源限制。我们相信我们的方法是可推广的，并为利用低功耗加速器和低功耗无线电铺平了道路。它可用于许多物联网和传感器网络部署。

我们的工作还开辟了几个研究方向。虽然我们专注于图像，但我们怀疑类似的方法也适用于复杂的音频处理模型，例如在噪声环境中启用语音处理的方法 [4]。此外，我们目前正在致力于将 Clio 扩展到视频分析模型——这些模型非常不同，因为它们通过 LSTMS、3D 卷积或 CNN 和光流跟踪的某种组合来学习时间依赖性 [20,22,43,54,57]。我们的期望是，随着这些模型复杂性的增加，边缘和云之间的划分变得更加重要，并且诸如 Clio 之类的模型编译方法将更加重要。

致谢

本文报告的研究部分由 CCDC 陆军研究实验室 (ARL) 根据合作协议 W911NF-17-2-0196 (ARL IoBT CRA) 以及国家自然科学基金会根据拨款号 1719386 和 1815347 赞助。本文件中包含的观点和结论仅代表作者的观点和结论，不应被解释为代表官方政策，无论是明示还是暗示。ARL、NSF 或美国政府。尽管此处有任何版权注释，美国政府仍有权出于政府目的的复制和分发重印本。

参考

- [1] cc2531 zigbee 芯片。 <http://www.ti.com/lit/ds/symlink/cc2531.pdf>.
- [2] Github 仓库。 <https://github.com/jinhuang01/CLIO>.
- [3] 采用 lora 技术的长距离、低功耗射频收发器 860-1000 mhz。 <https://www.semtech.com/products/wireless-rf/lora-transceivers/sx1272>.
- [4] Nsf: 耳戴式设备挑战。 <https://www.nasa.gov/feature/nsl-hearables-challenge>.
- [5] 圣sbt2632c2a蓝牙模块。 <http://www.st.com/web/en/resource/technical/document/datasheet/DM00048919.pdf>.
- [6] Stm32f7: 一款具有 Arm Cortex-M7 内核的高性能 MCU。 <https://www.st.com/en/microcontrollers-microprocessors/stm32f7-series.html>.
- [7] A. Ashok, P. Steenkiste 和 F. Bai. 通过自适应计算卸载，使用云服务启用车辆应用程序。第六届移动云计算和服务国际研讨会论文集，第 1-7 页。ACM, 2015.
- [8] S. Bhattacharya 和 N. D. Lane. 深度学习层的稀疏化和分离，用于可穿戴设备上的受限资源推理。第 14 届 ACM 嵌入式网络传感器系统会议记录 CD-ROM，第 176-189 页。ACM, 2016.
- [9] 博世。BMI160: 超低功耗惯性测量装置。

- [10] Y. Cheng, D. Wang, P. Zhou, and T. Zhang. A survey of model compression and acceleration for deep neural networks. *arXiv preprint arXiv:1710.09282*, 2017.
- [11] B.-G. Chun, S. Ihm, P. Maniatis, M. Naik, and A. Patti. Clonedcloud: elastic execution between mobile device and cloud. In *Proceedings of the sixth conference on Computer systems*, pages 301–314. ACM, 2011.
- [12] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein. *Introduction to algorithms*. MIT press, 2009.
- [13] E. Cuervo, A. Balasubramanian, D. Ki Cho, A. Wolman, S. Saroiu, R. Chandra, and P. Bahl. Maui: Making smartphones last longer with code offload. In *Proceedings of ACM MobiSys*, 2010.
- [14] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei. ImageNet: A Large-Scale Hierarchical Image Database. In *CVPR09*, 2009.
- [15] Y. Han, X. Wang, V. C. M. Leung, D. Niyato, X. Yan, and X. Chen. Convergence of edge computing and deep learning: A comprehensive survey. *CoRR*, abs/1907.08349, 2019.
- [16] K. He, X. Zhang, S. Ren, and J. Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 770–778, 2016.
- [17] Y. He, J. Lin, Z. Liu, H. Wang, L.-J. Li, and S. Han. Amc: Automl for model compression and acceleration on mobile devices. In *Proceedings of the European Conference on Computer Vision (ECCV)*, pages 784–800, 2018.
- [18] Himax. *HM01B0: Ultra Low Power Image Sensor*.
- [19] J. Huang, A. Badam, R. Chandra, and E. B. Nightingale. Weardrive: Fast and energy-efficient storage for wearables. In *2015 USENIX Annual Technical Conference (USENIX ATC 15)*, pages 613–625, Santa Clara, CA, July 2015. USENIX Association.
- [20] S. Ji, W. Xu, M. Yang, and K. Yu. 3d convolutional neural networks for human action recognition. *IEEE transactions on pattern analysis and machine intelligence*, 35(1):221–231, 2013.
- [21] Y. Kang, J. Hauswald, C. Gao, A. Rovinski, T. Mudge, J. Mars, and L. Tang. Neurosurgeon: Collaborative intelligence between the cloud and mobile edge. In *ACM SIGARCH Computer Architecture News*, volume 45, pages 615–629. ACM, 2017.
- [22] A. Karpathy, G. Toderici, S. Shetty, T. Leung, R. Sukthankar, and L. Fei-Fei. Large-scale video classification with convolutional neural networks. In *Proceedings of the IEEE conference on Computer Vision and Pattern Recognition*, pages 1725–1732, 2014.
- [23] E. C. Kevin Boos, David Chu. Flashback: Bringing immersive virtual reality to mobile devices through aggressive rendering memoization. In *Proceedings of the 14th Annual International Conference on Mobile Systems, Applications, and Services, MobiSys '16*, 2016.
- [24] J. H. Ko, T. Na, M. F. Amir, and S. Mukhopadhyay. Edge-host partitioning of deep neural networks with feature space encoding for resource-constrained internet-of-things platforms. In *2018 15th IEEE International Conference on Advanced Video and Signal Based Surveillance (AVSS)*, pages 1–6. IEEE, 2018.
- [25] A. Krizhevsky. Learning multiple layers of features from tiny images, 2009.
- [26] L. Lai and N. Suda. Enabling deep learning at the iot edge. In *Proceedings of the International Conference on Computer-Aided Design, ICCAD '18*, 2018.
- [27] L. Lai, N. Suda, and V. Chandra. Cmsis-nn: Efficient neural network kernels for arm cortex-m cpus. *arXiv preprint arXiv:1801.06601*, 2018.
- [28] N. D. Lane, P. Georgiev, and L. Qendro. Deepear: robust smartphone audio sensing in unconstrained acoustic environments using deep learning. In *Proceedings of the 2015 ACM International Joint Conference on Pervasive and Ubiquitous Computing*, pages 283–294. ACM, 2015.
- [29] H. Li, C. Hu, J. Jiang, Z. Wang, Y. Wen, and W. Zhu. Jalad: Joint accuracy-and latency-aware deep structure decoupling for edge-cloud execution. In *2018 IEEE 24th International Conference on Parallel and Distributed Systems (ICPADS)*, pages 671–678. IEEE, 2018.
- [30] F. Liu, P. Shu, H. Jin, L. Ding, J. Yu, D. Niu, and B. Li. Gearing resource-poor mobile devices with powerful clouds: architectures, challenges, and applications. *Wireless Communications, IEEE*, 20(3):14–22, 2013.
- [31] S. Liu, Y. Lin, Z. Zhou, K. Nan, H. Liu, and J. Du. On-demand deep model compression for mobile devices: A usage-driven model selection framework. 2018.
- [32] Z. Liu, T. Liu, W. Wen, L. Jiang, J. Xu, Y. Wang, and G. Quan. Deepn-jpeg: a deep neural network favorable jpeg-based image compression framework. In *Proceedings of the 55th Annual Design Automation Conference*, pages 1–6, 2018.
- [33] Z. Liu, H. Mu, X. Zhang, Z. Guo, X. Yang, K.-T. Cheng, and J. Sun. Metapruning: Meta learning for automatic neural network channel pruning. In *Proceedings of the IEEE International Conference on Computer Vision*, pages 3296–3305, 2019.
- [34] H. Mao, R. Netravali, and M. Alizadeh. Neural adaptive video streaming with pensieve. In *Proceedings of the Conference of the ACM Special Interest Group on Data Communication*, pages 197–210, 2017.
- [35] Maxim Integrated. *Ultra-Low Power, Single-Channel Integrated Biopotential (ECG, R to R Detection) AFE*.
- [36] Nordic Semiconductor. *NRF51822: Bluetooth low energy and 2.4GHz proprietary SoC*.
- [37] H. Riiser, T. Endestad, P. Vigmostad, C. Griwodz, and P. Halvorsen. Video streaming using a location-based bandwidth-lookup service for bitrate planning. *ACM Transactions on Multimedia Computing, Communications, and Applications (TOMM)*, 8(3):24, 2012.
- [38] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen. Mobilenetv2: Inverted residuals and linear bottlenecks. In *2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 4510–4520. IEEE, 2018.
- [39] M. Satyanarayanan. A brief history of cloud offload: A personal journey from odyssey through cyber foraging to cloudlets. *GetMobile: Mobile Computing and Communications*, 18(4):19–23, 2015.
- [40] W. Shi, Y. Hou, S. Zhou, Z. Niu, Y. Zhang, and L. Geng. Improving device-edge cooperative inference of deep learning via 2-step pruning. *arXiv preprint arXiv:1903.03472*, 2019.
- [41] W. Shi, Y. Hou, S. Zhou, Z. Niu, Y. Zhang, and L. Geng. Improving device-edge cooperative inference of deep learning via 2-step pruning. *CoRR*, abs/1903.03472, 2019.
- [42] K. Simonyan and A. Zisserman. Very deep convolutional networks for large-scale image recognition. *arXiv preprint arXiv:1409.1556*, 2014.
- [43] L. Sun, K. Jia, D.-Y. Yeung, and B. E. Shi. Human action recognition using factorized spatio-temporal convolutional networks. In *Proceedings of the IEEE International Conference on Computer Vision*, pages 4597–4605, 2015.
- [44] S. Teerapittayanon, B. McDanel, and H.-T. Kung. Branchynet: Fast inference via early exiting from deep neural networks. In *2016 23rd International Conference on Pattern Recognition (ICPR)*, pages 2464–2469. IEEE, 2016.
- [45] S. Teerapittayanon, B. McDanel, and H.-T. Kung. Distributed deep neural networks over the cloud, the edge and end devices. In *2017 IEEE 37th International Conference on Distributed Computing Systems (ICDCS)*, pages 328–339. IEEE, 2017.
- [46] J. Tosi, F. Taffoni, M. Santacatterina, R. Sannino, and D. Formica. Performance evaluation of bluetooth low energy: a systematic review. *Sensors*, 17(12):2898, 2017.
- [47] <https://aws.amazon.com/sagemaker/neo/>. Amazon SageMaker Neo: Train models once, run anywhere with up to 2x performance improvement.
- [48] <https://developers.google.com/ml-kit/>. Google ML Kit: Bringing Google's ML expertise to mobile developers.
- [49] https://en.wikipedia.org/wiki/IEEE_802.11ah. 802.11ah WiFi HaLow: WiFi for Internet of Things.
- [50] <https://etacompute.com/>. Etacompute TENSAT: Machine Learning Platform with Ultra-Low-Power Consumption for Edge Devices.
- [51] <https://greenwaves-technologies.com/gap8-product/>. GAP8: Ultra-low power, always-on processor for embedded artificial intelligence.
- [52] <https://www.syntiant.com/>. SYNTIANT: Always-on machine learning solutions for battery-powered devices.
- [53] <https://www.tensorflow.org/lite>. Tensor Flow Lite: Deploy machine learning models on mobile and IoT devices.
- [54] G. Varol, I. Laptev, and C. Schmid. Long-term temporal convolutions for action recognition. *IEEE transactions on pattern analysis and machine intelligence*, 40(6):1510–1517, 2018.
- [55] Vesper. *VM1010: Wake-on-Sound Piezoelectric MEMS Microphone*.
- [56] S. Yao, Y. Zhao, H. Shao, S. Liu, D. Liu, L. Su, and T. Abdelzaher. Fastdeepiot: Towards understanding and optimizing neural network execution time on mobile and embedded devices. In *Proceedings of the 16th ACM Conference on Embedded Networked Sensor Systems*, pages 278–291. ACM, 2018.
- [57] J. Yue-Hei Ng, M. Hausknecht, S. Vijayanarasimhan, O. Vinyals, R. Monga, and G. Toderici. Beyond short snippets: Deep networks for video classification. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 4694–4702, 2015.

- [10] Y. Cheng, D. Wang, P. Zhou, 和 T. Zhang. 神经网络模型压缩和加速的调查. arXiv 预印本 arXiv:1710.09282, 2017. [11] B.-G. Chun, S. Ihm, P. Maniatis, M. Naik 和 A. Patti. Clonecloud: 移动设备和云之间的弹性执行. 第六届计算机系统会议论文集, 第 301-314 页. ACM, 2011. [12] T. H. Cormen, C. E. Leiserson, R. L. Rivest 和 C. Stein. 算法简介. 麻省理工学院出版社, 2009 年. [13] E. Cuervo, A. Balasubramanian, D. ki Cho, A. Wolman, S. Saroiu, R. Chandra 和 P. Bahl. 毛伊岛: 通过代码卸载使智能手机的使用寿命更长. 收录于 ACM MobiSys 论文集, 2010 年. [14] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li 和 L. Fei-Fei. ImageNet: 大规模分层图像数据库. CVPR09, 2009. [15] Y. Han, X. Wang, V. C. M. Leung, D. Niyato, X. Yan, X. Chen. 边缘计算和深度学习的融合: 综合调查. CoRR, abs/1907.08349, 2019. [16] 何坤, 张旭, 任胜, 孙杰. 用于图像识别的深度残差学习. IEEE 计算机视觉和模式识别会议论文集, 第 770-778 页, 2016 年. [17] Y. He, J. Lin, Z. Liu, H. Wang, L.-J. 李和 S. Han. Amc: 用于移动设备上的模型压缩和加速的 Automl. 欧洲计算机视觉会议 (ECCV) 会议记录, 第 784-800 页, 2018 年. [18] Himax. HM01B0: 超低功耗图像传感器. [19] J. Huang, A. Badam, R. Chandra 和 E. B. Nightingale. Weardrive: 快速、节能的可穿戴设备存储. 2015 年 USENIX 年度技术会议 (USENIX ATC 15), 第 613-625 页, 加利福尼亚州圣克拉拉, 2015 年 7 月. USENIX 协会. [20] 吉树, 徐伟, 杨明, 余坤. 用于人类动作识别的 3D 卷积神经网络. IEEE 模式分析和机器智能交易, 35(1):221-231, 2013. [21] Y. Kang, J. Hauswald, C. Gau, A. Rovinski, T. Mudge, J. Mars 和 L. Tang. 神经外科医生: 云和移动边缘之间的协作智能. 摘自 ACM SIGARCH 计算机架构新闻, 第 45 卷, 第 615-629 页. ACM, 2017. [22] A. Karpathy, G. Toderici, S. Shetty, T. Leung, R. Sukthankar 和 L. Fei-Fei. 使用卷积神经网络进行大规模视频分类. IEEE 计算机视觉和模式识别会议论文集, 第 1725-1732 页, 2014 年. [23] E. C. Kevin Boos, David Chu. Flashback: 通过积极的渲染记忆将沉浸式虚拟现实引入移动设备. 第 14 届移动系统、应用程序和服务国际年会议论文集, MobiSys '16, 2016. [24] J. H. Ko, T. Na, M. F. Amir 和 S. Mukhopadhyay. 针对资源受限的物联网平台, 使用特征空间编码对神经网络进行边缘主机分区. 2018 年第 15 届 IEEE 国际高级视频和信号监控 (AVSS) 会议, 第 1-6 页. IEEE, 2018. [25] A. Krizhevsky. 从微小图像中学习多层特征, 2009 年. [26] L. Lai 和 N. Suda. 在物联网边缘启用深度学习. 国际计算机辅助设计会议记录, ICCAD '18, 2018. [27] L. Lai, N. Suda 和 V. Chandra. Cmsis-nn: 适用于 Arm Cortex-M CPU 的高效神经网络内核. arXiv 预印本 arXiv:1801.06601, 2018. [28] N. D. Lane, P. Georgiev 和 L. Qendro. Deeppear: 使用深度学习在不受约束的声学环境中实现强大的智能手机音频传感. 2015 年 ACM 国际普适计算联合会议论文集, 第 283-294 页. ACM, 2015. [29] 李海, 胡春, 江建, 王志, 温勇, 朱伟. Jalad: 用于边缘云执行的联合准确性和延迟感知深度结构解耦. 2018 年 IEEE 第 24 届并行与分布式系统国际会议 (ICPADS), 第 671-678 页. IEEE, 2018. [30] F. Liu, P. Shu, H. Jin, L. Ding, J. Yu, D. Niu 和 B. Li. 使用强大的云来适应资源匮乏的移动设备: 架构、挑战和应用程序. 无线通信, IEEE, 20(3):14-22, 2013. [31] S. Liu, Y. Lin, Z. Zhou, K. Nan, H. Liu 和 J. Du. 移动设备的按需深度模型压缩: 使用驱动模型选择框架. 2018. [32] 刘子, 刘涛, 温文, 蒋丽, 徐建, 王勇, 权国强. Deepn-jpeg: 一种适合深度神经网络的基于 jpeg 的图像压缩框架. 第 55 届年度设计自动化会议论文集, 第 1-6 页, 2018 年. [33] Z. Liu, H. Mu, X. 张, Z. 郭, X. Yang, K.-T. 程和孙杰. 元剪枝: 用于自动神经网络通道剪枝的元学习. IEEE 国际计算机视觉会议论文集, 第 3296-3305 页, 2019 年. [34] H. Mao, R. Netravali 和 M. Alizadeh. 使用冥想盆进行神经自适应视频流. 载于 ACM 数据通信特别兴趣小组会议记录, 第 197-210 页, 2017 年. [35] Maxim Integrated. 超低功耗、单通道集成生物电势 (E CG、R 至 R
- ACM 多媒体计算、通信和应用汇刊 (TOMM), 8(3):24, 2012. [38] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov 和 L.-C. 陈. Mobilenetv2: 反转残差和线性瓶颈. 2018 年 IEEE/CVF 计算机视觉和模式识别会议, 第 4510-4520 页. IEEE, 2018. [39] M. Satyanarayanan. 云卸载简史: 从奥德赛到网络搜索再到云的个人旅程. GetMobile: 移动计算与通信, 18(4):19-23, 2015. [40] W. Shi, Y. Hou, S. Zhou, Z. Niu, Y. Zhang, 和 L. Geng. 通过两步剪枝改进深度学习的设备边缘协作推理. arXiv 预印本 arXiv:1903.03472, 2019. [41] W. Shi, Y. Hou, S. Zhou, Z. Niu, Y. Zhang 和 L. Geng. 通过两步剪枝改进深度学习的设备边缘协作推理. CoRR, abs/1903.03472, 2019. [42] K. Simonyan 和 A. Zisserman. 用于大规模图像识别的非常深的卷积网络. arXiv 预印本 arXiv:1409.1556, 2014. [43] L. Sun, K. Jia, D.-Y. 杨和 B. E. Shi. 使用因式分解时空卷积网络进行人类动作识别. IEEE 国际计算机视觉会议论文集, 第 4597-4605 页, 2015 年. [44] S. Teerapittayanon, B. McDanel 和 H.-T. 恭. Branchynet: 通过早期退出神经网络进行快速推理. 2016 年第 23 届国际模式识别会议 (ICPR), 第 2464-2469 页. IEEE, 2016. [45] S. Teerapittayanon, B. McDanel 和 H.-T. 恭. 云端、边缘和终端设备上的分布式深度神经网络. 2017 年 IEEE 第 37 届分布式计算系统国际会议 (ICDCS), 第 328-339 页. IEEE, 2017. [46] J. Tosi, F. Taffoni, M. Santacatterina, R. Sannino 和 D. Formica. 蓝牙低功耗性能评估: 系统评价. 传感器, 17(12):2898, 2017. [47] <https://aws.amazon.com/sagemaker/neo/>. Amazon SageMaker Neo: 模型训练一次, 可在任何地方运行, 性能提升高达 2 倍. [48] <https://developers.google.com/ml-kit/>. Google ML Kit: 将 Google 的 ML 专业知识带给移动开发者. [49] https://en.wikipedia.org/wiki/IEEE_802.11ah. 802.11ah WiFi HaLow: 物联网 WiFi. [50] <https://etacompute.com/>. Etacompute TENSAT: 适用于边缘设备的超低功耗机器学习平台. [51] <https://greenwaves-technologies.com/gap8-product/>. GAP8: 用于嵌入式人工智能的超低功耗、始终在线的处理器. [52] <https://www.syntiant.com/>. SYNTIANT: 适用于电池供电设备的始终在线机器学习解决方案. [53] <https://www.tensorflow.org/lite>. Tensor Flow Lite: 在移动和物联网设备上部署机器学习模型. [54] G. Varol, I. Laptev 和 C. Schmid. 用于动作识别的长期时间卷积. IEEE 模式分析和机器智能交易, 40(6):1510-1517, 2018. [55] Vesper. VM1010: 声音唤醒压电 MEMS 麦克风. [56] S. Yao, Y. Zhu, H. Shao, S. Liu, D. Liu, L. Su, T. Abdelzaher. Fastdeepiot: 了解和优化移动和嵌入式设备上的神经网络执行时间. 第 16 届 ACM 嵌入式网络传感器系统会议论文集, 第 278-291 页. ACM, 2018. [57] J. Yue-Hei Ng, M. Hausknecht, S. Vijayanarasimhan, O. Vinyals, R. Monga 和 G. Toderici. 超越简短片段: 用于视频分类的深度网络. IEEE 计算机视觉和模式识别会议论文集, 第 4694-4702 页, 2015 年.